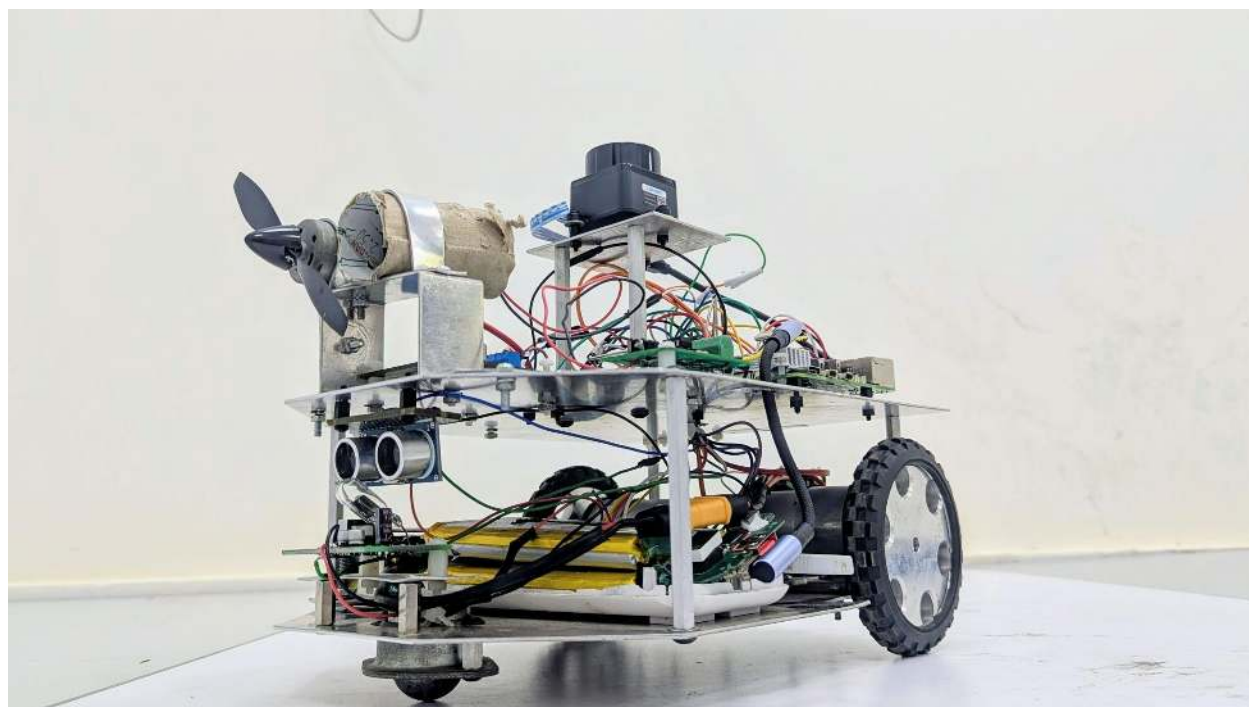


פרויקט גמר במגמת רובוטיקה - 5 יח"ל



מגישי הפרויקט: אביתר מוצ'ניק, גלעד ברקוב וראובן למברג

331443515 - גלעד ברקוב
217510403 - אביתר מוצ'ניק
218086114 - ראובן למברג



תוכן עניינים

מבוא	3
הגדרת הבעיה ורקע על התחרות	4
מבנה הרובוט ומערכות ההנעה	10
סיכום רכיבי הרובוט	13
מערכת חשמל והספק	19
הסבר חיישני הרובוט	22
תכנות	25
רפלקציות אישיות	64
ניתוח הנדסי והסבר מבני(שרטוטים)	67
מקורות	75



מבוא

בשנים האחרונות תחום הרובוטיקה האוטונומית מתפתח בקצב מהיר והופך לחלק משמעותי מעולם הטכנולוגיה המודרני. רובוטים אוטונומיים משמשים כיום בתחומים רבים כגון תעשייה, רפואה, ביטחון, תחבורה, חקר החלל ואף במוצרים ביתיים כמו שואבים רובוטיים ורכבים חכמים. מטרתם של רובוטים אלו היא לבצע משימות שונות באופן עצמאי, תוך קבלת מידע מהסביבה, עיבוד הנתונים וקבלת החלטות בזמן אמת ללא התערבות אנושית ישירה.

במסגרת פרויקט זה בנינו רובוט אוטונומי המשתתף בתחרות RoboNer. מטרת התחרות היא לבנות רובוט עצמאי המסוגל לנווט בזירה המדמה מבנה סגור, לאתר מקור אש המדומה באמצעות נר, לכבות אותו ולחזור לנקודת ההתחלה בזמן הקצר ביותר. במהלך המשימה הרובוט נדרש להתמודד עם מכשולים, מעברים וחדרים שונים, תוך שמירה על אמינות, דיוק ויכולת קבלת החלטות בזמן אמת.

הרובוט שבנינו מבוסס על שילוב בין Raspberry Pi 5 לבין Arduino Uno, אשר עובדים יחד באמצעות תקשורת UART. מבנה זה מאפשר חלוקת עבודה בין מערכת עיבוד הנתונים לבין מערכת בקרת החומרה, ובכך משפר את יציבות המערכת ואת ביצועי הרובוט.

המאפיין המרכזי והייחודי ביותר ברובוט שלנו הוא השימוש ב-LiDAR sensor כחיישן הניווט הראשי של הרובוט. בניגוד לרובוטים פשוטים יותר המשתמשים בעיקר בחיישני מרחק בסיסיים, חיישן ה-LiDAR מאפשר לרובוט לבצע סריקה מדויקת של הסביבה באמצעות לייזר, לזהות קירות, פתחים ומכשולים, ולנווט בצורה מדויקת ואמינה יותר בזירה. טכנולוגיה זו דומה לטכנולוגיות המשמשות כיום בשואבים רובוטיים, רכבים אוטונומיים ורובוטים תעשייתיים מתקדמים.

בנוסף למערכת הניווט, הרובוט כולל מערכת כיבוי אש המבוססת על מאוורר, חיישן UV לזיהוי הימצאות להבה בחדר, וחיישן נוסף המשמש למציאת מיקומו המדויק של הנר לצורך כיבוי יעיל. כלל המערכות פועלות באופן אוטונומי מלא, כאשר לרובוט קיימת גם אפשרות להפעלה ידנית מרחוק לצורכי בדיקות ופיתוח.

במהלך העבודה על הפרויקט נדרשנו להתמודד עם אתגרים רבים בתחומי התוכנה, האלקטרוניקה, הבקרה והמכניקה. תהליך הפיתוח כלל תכנון מערכת החשמל, בניית שלדת הרובוט, כתיבת אלגוריתמים לניווט אוטונומי, שילוב חיישנים שונים וביצוע בדיקות רבות לשיפור אמינות המערכת.

מעבר לצד הטכנולוגי, מטרות הפרויקט היו גם לצבור ניסיון מעשי בתחומי ההנדסה והרובוטיקה, ללמוד לעבוד בצוות, להתמודד עם אתגרים מורכבים, לשפר את יכולות פתרון הבעיות שלנו ולחוות תהליך פיתוח אמיתי של מערכת רובוטית מתקדמת. בנוסף, הפרויקט אפשר לנו להעמיק בתחומים שלא נלמדים בצורה מעשית במסגרת הלימודים הרגילה, תוך שיתוף פעולה ועבודה קבוצתית לאורך זמן.

בעבודה זו נציג את שלבי הפיתוח של הרובוט, מבנהו המכני והאלקטרוני, מערכות ההנעה, החיישנים, מערכת התוכנה והבקרה, האתגרים המרכזיים שעמדו בפנינו והפתרונות שפיתחנו במהלך בניית הרובוט.



הגדרת הבעיה ורקע כללי על התחרות

תחרות RoboNer היא תחרות רובוטיקה אוטונומית המתקיימת מדי שנה ומיועדת לתלמידים וסטודנטים מתחום הרובוטיקה, האלקטרוניקה והתוכנה. מטרת התחרות היא לבנות רובוט אוטונומי המסוגל לבצע משימת חילוץ וכיבוי אש בתוך זירה המדמה מבנה סגור. במהלך התחרות הרובוט נדרש לנוע באופן עצמאי לחלוטין בתוך הזירה, לזהות את מיקומו, לאתר מקור אש המדומה באמצעות נר, לכבות אותו ולחזור לנקודת ההתחלה — וכל זאת ללא שליטה אנושית בזמן הריצה.

מטרת התחרות אינה רק לבדוק האם הרובוט מסוגל לנוע, אלא לבחון את רמת הדיוק, האמינות, מהירות התגובה, איכות התוכנה, יכולת ההתמודדות עם בעיות בזמן אמת ורמת התכנון ההנדסי של הקבוצה. לכן מדובר בפרויקט המשלב מספר רב של תחומים:

- תכנות
- אלקטרוניקה
- מכניקה
- בקרה
- ניווט אוטונומי
- עיבוד מידע מחיישנים
- מערכות חשמל
- עבודה צוותית ותכנון הנדסי

הזירה בתחרות בנויה בצורה המדמה מבנה אמיתי הכולל מסדרונות, חדרים, קירות, פניות חדות ולעיתים גם מכשולים שונים. הרובוט מתחיל מנקודת הזינוק ועליו למצוא באופן עצמאי את הדרך בתוך המבוך עד לחדר שבו נמצא הנר. לאחר זיהוי הנר על הרובוט להתקרב אליו, לכבות אותו באמצעות מערכת הכיבוי המותקנת עליו ולאחר מכן למצוא את הדרך חזרה אל נקודת ההתחלה.

במבט ראשון משימה זו עשויה להיראות פשוטה יחסית, אך בפועל מדובר באחת המשימות המורכבות ביותר בתחום הרובוטיקה האוטונומית. הסיבה לכך היא שהרובוט חייב לבצע מספר רב של פעולות בזמן ובזמן אמת:

- לזהות את סביבתו.
- להבין את מיקומו במרחב.
- להימנע מהתנגשויות.
- לקבל החלטות ניווט.
- לזהות פתחים וחדרים.
- לאתר מקור אש.
- לבצע כיבוי בצורה מדויקת.
- ולחזור חזרה לנקודת ההתחלה ללא טעויות.



כל אחת מהפעולות הללו בפני עצמה נחשבת למשימה מורכבת בתחום הרובוטיקה. כאשר משלבים את כולן במערכת אחת הפועלת בזמן אמת ובאופן עצמאי לחלוטין, רמת הקושי עולה בצורה משמעותית מאוד.

בנוסף לכך, חוקי התחרות מגבילים מאוד את אופן פעולת הרובוט. על פי חוקי התחרות הרובוט חייב להיות אוטונומי לחלוטין במהלך הריצה. כלומר, לאחר תחילת המשימה אסור למפעיל להתערב בפעולת הרובוט או לשלוט בו. הרובוט חייב לקבל החלטות בעצמו ולהגיב לסביבה באופן עצמאי. עובדה זו היא אחת הסיבות המרכזיות לכך שהרובוט נקרא "רובוט אוטונומי".

לרובוט שלנו קיימת גם אפשרות הפעלה ידנית לצורכי פיתוח ובדיקות, באמצעות שליטה מרחוק מהמחשב, אך בזמן התחרות הוא פועל במצב אוטונומי מלא בלבד.

מעבר לדרישת האוטונומיה, קיימות בתחרות מגבלות נוספות כגון:

- מגבלת גודל ומשקל לרובוט.
- מגבלות בטיחות.
- איסור על חיבור קבוע לחשמל.
- דרישה למערכת כיבוי בטוחה.
- דרישה לכפתור חירום.
- מגבלות זמן לביצוע המשימה.
- וחובת פעולה אמינה לאורך כל הריצה.

משמעות הדבר היא שלא מספיק שהרובוט "יעבוד לפעמים", אלא עליו לפעול בצורה יציבה ואמינה כמעט בכל ניסיון. גם אם מערכת מסוימת מצליחה לפעול היטב בחלק מהמקרים אך נכשלת לעיתים קרובות — הדבר עלול לגרום לכישלון מוחלט בתחרות.

אחד האתגרים המרכזיים ביותר בפרויקט שלנו היה מערכת הניווט של הרובוט. בניגוד לרובוטים פשוטים יותר, אשר משתמשים במספר קטן של חיישני מרחק בסיסיים, אנו החלטנו לבנות רובוט המבוסס על LiDAR sensor כחיישן הניווט המרכזי.

החלטה זו הפכה את הפרויקט למתקדם ומדויק יותר, אך גם למורכב משמעותית. בעוד שחיישני מרחק רגילים כגון חיישני IR או חיישנים אולטראסוניים מספקים מספר קטן יחסית של נקודות מידע, חיישן LiDAR מסוגל לבצע סריקה רחבה של הסביבה באמצעות קרני לייזר ולספק לרובוט תמונה דו-ממדית מלאה של החדר והמסדרונות שסביבו.

באמצעות מידע זה ניתן ליצור מפה של הזירה, לזהות קירות ופתחים, להבין את מבנה הסביבה ולבצע ניווט מדויק הרבה יותר. למעשה, טכנולוגיה זו דומה לטכנולוגיות הקיימות כיום בשואבים רובוטיים, רכבים אוטונומיים ורובוטים תעשייתיים מתקדמים.



הסיבה המרכזית שבגללה בחרנו להשתמש ב-LiDAR הייתה הצורך לקבל מידע רב ומדויק על הסביבה. אם היינו משתמשים בחיישנים רגילים בלבד, לא היינו מקבלים מספיק מידע כדי לבנות מפה אמינה של הזירה. בנוסף, חיישנים רגילים מושפעים יותר מהחזרים, רעשים ותנאי סביבה שונים, בעוד שחיישן LiDAR מספק מידע מדויק ויציב יותר.

יתרון נוסף של ה-LiDAR הוא העובדה שהוא מאפשר לקבל מידע אודומטרי על מיקום הרובוט גם ללא שימוש באנקודרים על המנועים. ברובוט שלנו לא השתמשנו באנקודרים, ולכן מערכת ה-LiDAR הייתה חיונית להבנת מיקום הרובוט וליכולת הניווט שלו.

עם זאת, השימוש ב-LiDAR יצר את אחד האתגרים הקשים ביותר בפרויקט כולו. הקושי הראשון היה עצם קבלת המידע מהחיישן. בניגוד לחיישנים פשוטים שמעבירים ערך בודד של מרחק, חיישן ה-LiDAR מספק כמות גדולה מאוד של מידע בזמן אמת, ולכן היה צורך ללמוד כיצד לקרוא את הנתונים ולעבד אותם בצורה נכונה.

לאחר מכן נדרשנו להתמודד עם חיבור של טרנספורמציות תלת-ממדיות בין מערכות קואורדינטות שונות ברובוט. היה צורך לגרום לכל מערכות המיקום והמדידה לעבוד יחד בצורה מסונכרנת ומדויקת. תהליך זה היה מורכב מאוד משום שכל סטייה קטנה במיקום או בזווית גרמה לשגיאות מצטברות במפה ובניווט.

לאחר חיבור המידע מה-LiDAR נדרשנו לשלב אותו יחד עם מערכת האודומטריה של הרובוט. האודומטריה היא המערכת שמאפשרת לרובוט להעריך את מיקומו ביחס לסביבה שלו. לאחר מכן היה צורך להעביר את המידע למערכת SLAM, אשר אחראית על יצירת מפה של הזירה תוך כדי תנועה.

מערכת SLAM היא אחת המערכות המורכבות ביותר בתחום הרובוטיקה האוטונומית, משום שהיא דורשת מהרובוט גם להבין היכן הוא נמצא וגם לבנות מפה של הסביבה באותו הזמן. תהליך זה דרש מאיתנו זמן רב של ניסויים, תיקונים ושיפורים.

גם לאחר שהצלחנו ליצור מפות בסיסיות של הזירה, עדיין נדרשנו להתמודד עם בעיית האמינות. בתחילת הדרך הרובוט לא הצליח ליצור את אותה המפה בצורה עקבית בכל ריצה. לעיתים נוצרו שגיאות

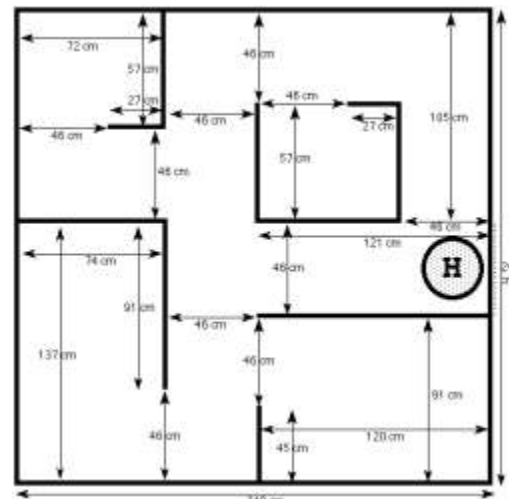
במיקום, ולעיתים הרובוט איבד את מיקומו לחלוטין. לכן היינו צריכים לבצע שיפורים רבים באלגוריתמים ובאופן עיבוד המידע עד שהצלחנו להגיע לרמת אמינות מספקת.

לאחר פתרון בעיית המיפוי התחיל שלב מורכב נוסף — פיתוח מערכת הניווט עצמה. בשלב זה הרובוט כבר ידע ליצור מפה של הסביבה, אך עדיין היה צורך לגרום לו להבין כיצד להשתמש במפה לצורך תכנון מסלול, מעבר בין חדרים, פנייה במסדרונות והימנעות ממכשולים.



הניווט עצמו היה האתגר המרכזי בתחרות מבחינתנו. כל טעות קטנה בניווט עלולה לגרום לרובוט:

- להיכנס בקיר.
- לפספס חדר.
- לאבד את מיקומו.
- להיתקע.
- או לא למצוא את הדרך חזרה.



בנוסף לקשיי הניווט, גם תחום התוכנה מהווה אתגר עצום. הפרויקט שלנו מבוסס על שילוב של Raspberry Pi עם Arduino Uno, אשר מתקשרים ביניהם באמצעות UART.

ה-Raspberry Pi אחראי על עיבוד נתוני ה-LiDAR, מערכת הניווט והמיפוי, בעוד שה-Arduino אחראי על בקרת המנועים ורכיבי החומרה. שילוב בין שני בקרים שונים דרש מאיתנו לבנות מערכת תקשורת אמינה ומהירה, שתאפשר סנכרון מלא בין כל חלקי הרובוט.

בנוסף, הפרויקט נכתב בסביבת Linux תוך שימוש בשפת Python. מכיוון שמדובר בפרויקט מורכב מאוד, תהליך התכנות היה מבוסס פעמים רבות על ניסוי וטעייה. במקרים רבים השקענו זמן רב בפתרון בעיה מסוימת, ורק לאחר בדיקות רבות הצלחנו להבין מה מקור התקלה.



גם מערכת החשמל הייתה אתגר משמעותי מאוד בפרויקט. הרובוט מבוסס על שתי סוללות שונות:

- סוללה אחת עבור מערכת העיבוד וה-LiDAR.
- וסוללה נוספת עבור שאר מערכות הרובוט.

בנוסף בנינו מפצל מתחים ייעודי אשר מקבל מתח של 12V ומפיק מתחים שונים לרכיבי הרובוט:
12V, 8V, 5V, GND

מערכת זו הייתה מורכבת מאוד משום שהיה צורך לספק מתחים שונים לרכיבים שונים ברובוט, תוך שמירה על יציבות המערכת ומניעת רעשים חשמליים.

בתחילת הפרויקט בנינו מפצל מתחים ראשון, אך לאחר זמן רב של עבודה גילינו שהמערכת אינה יציבה מספיק ואינה מתאימה לצרכים של הרובוט. בעקבות זאת נאלצנו לפרק חלקים משמעותיים מהרובוט, לבנות מחדש את מערכת החשמל ולתכנן מפצל מתחים חדש לחלוטין. תהליך זה דרש עבודה רבה מאוד וגרם לעיכובים משמעותיים בפרויקט.

גם מערכת הכיבוי עצמה הייתה מורכבת יחסית. הרובוט משתמש בחיישן UV לצורך זיהוי הימצאות להבה בחדר. לאחר זיהוי הלהבה מופעל חיישן נוסף אשר משמש כ"כוונת" ומטרתו לזהות את מיקומו המדויק של הנר. לאחר איתור הנר הרובוט מפעיל מאוורר אשר מכבה את האש.

בנוסף לאתגרים הטכנולוגיים, הפרויקט דרש גם עבודה צוותית מורכבת. הפרויקט בוצע על ידי שלושה תלמידים במקביל ללימודים, מבחנים, עבודות ומחויבויות נוספות. לעיתים היה קשה לבצע סנכרון מלא בין כל חברי הקבוצה, במיוחד בפרויקט כה מורכב הכולל תוכנה, חומרה, חשמל ומכניקה. למרות זאת הצלחנו לעבוד יחד, לחלק תחומי אחריות ולשתף פעולה לאורך כל תהליך הפיתוח.

למעשה, אחד הדברים המרכזיים שלמדנו במהלך הפרויקט הוא שבניית רובוט מתקדם אינה תהליך ליניארי. פעמים רבות בנינו מערכת מסוימת ורק לאחר בדיקות רבות גילינו שיש צורך לפרק אותה ולבנות אותה מחדש בצורה שונה לחלוטין. מכיוון שמדובר ברובוט ברמה שלא נבנתה בעבר במסגרת התחרות בארץ, חלק גדול מהפרויקט התבסס על ניסויים, בדיקות, טעויות ושיפורים מתמשכים.

למרות כל הקשיים והאתגרים, הפרויקט אפשר לנו ללמוד בצורה מעשית תחומים מתקדמים מאוד בעולם הרובוטיקה וההנדסה, לצבור ניסיון אמיתי בעבודה על מערכת מורכבת, להתמודד עם בעיות ברמה גבוהה ולפתח יכולות טכניות ותוכניות שלא היינו נחשפים אליהן במסגרת לימודים רגילה.

במהלך שנת הפעילות הנוכחית, נאלצנו להתמודד עם אילוף חיצוני משמעותי ובלתי צפוי בדמות פריצת הלחימה במדינה במחצית הראשונה של שנת 2026, אירוע חירום לאומי אשר השפיע ישירות על קצב התקדמות הפרויקט, רציפותו ואופן מימוש יעדיו המקוריים. המצב הביטחוני והנחיות פיקוד העורף מנעו



מאיתנו גישה סדירה למעבדת הרובוטיקה של בית הספר, שבה נמצאים כלי העבודה, רכיבי החומרה ושטח הכיול המרכזי, ואילו קשה זה אילץ את הצוות לשנות לחלוטין את אסטרטגיית הפעולה ולעבור למתכונת עבודה מבוזרת מרחוק. תחת מגבלות אלו, הסתמכנו בצורה מוגברת על מערכות לניהול גרסאות קוד (Git) ועל סביבות סימולציה ממוחשבות, כאשר משימות התוכנה, כתיבת ארכיטקטורת ה-ROS2 וכיול אלגוריתמי ה-SLAM קודמו מתוך בטינו תוך שמירה על תקשורת רציפה. אף על פי שתחרות ה"רובונר" הארצית - שהיוותה את תכלית הפרויקט ומטרתו המרכזית - בוטלה בסופו של דבר עקב המצב ולא יכלה להתקיים במתכונתה המתוכננת, קיבלנו החלטה הנדסית נחרצת שלא לעצור את תהליך הפיתוח, ובמקום זאת בחרנו להתייחס לתקנון הרשמי של התחרות כאל "ספר דרישות לקוח" מחייב לכל דבר ועניין. כתוצאה מכך, המערכת הרובוטית הושלמה במלואה, עברה תהליכי אינטגרציה

קפדניים, והוכיחה יכולות עבודה מרשימות בתנאי מעבדה ובסביבות מדומות שבהן כוילו בהצלחה הלידאר, החיישן האולטראסוני, ה-IMU וחיישן הלהבה, לצד אימות יכולת ההזנקה האוטונומית באמצעות חיישן המיקרופון בהתאם לתדר האזעקה הנדרש. ההתמודדות המורכבת הזו עם אילוצי מלחמת 2026 אילצה אותנו לפעול כמהנדסים בתעשייה האמיתית הנדרשים להתמודד עם משברים בלתי צפויים, ובכך פיתחה בקרב חברי הצוות גמישות מחשבתית, יכולת ניהול פרויקטים במצבי אי-וודאות ופתרון בעיות תחת לחץ, תכונות המהוות נדבך קריטי להמשך דרכנו המקצועית והנדסית, ואנו גאים בתוצר הסופי שהצלחנו להעמיד אשר עונה על כל הדרישות המחמירות של מגמת הרובוטיקה



מבנה הרובוט ומערכות ההנעה

1. מבנה הרובוט והשלדה (Chassis Design)

שלדת הרובוט מהווה את התשתית הפיזית הנושאת את כלל מערכות הכוח, העיבוד, החישה והכיבוי. בניגוד לרובוטים מסחריים או ערכות מדף מוכנות מראש, שלדת הרובוט בפרויקט זה תוכננה ונבנתה מאפס (Custom Manufacturing) בהתאמה מלאה לאילוצי הנפח הקשיחים של תקנון תחרות הרובונר (31*31*31 ס"מ).

השלדה נבנתה באופן ידני מרצועות אלומיניום תעופתי. רצועות אלו עברו תהליכי עיבוד מכני שכללו חיתוך מבוקר באמצעות מספרי פח ייעודיים, כיפוף קונסטרוקטיבי, וקדיחת חורי קיבוע ועיגון עבור רכיבי החומרה והחיישנים השונים. הבחירה באלומיניום כחומר הגלם המרכזי למבנה הרובוט נשענת על מספר שיקולים הנדסיים מובהקים:

- **יחס חוזק-משקל (Strength-to-Weight Ratio):** אלומיניום הוא מתכת קלה במיוחד המאפשרת להפחית באופן משמעותי את המסה הכוללת ואת מומנט ההתמד (אינרציה) של הרובוט. הפחתה זו חוסכת באנרגיית סוללה ומאפשרת האצה ובלימה מהירות. מנגד, החומר קשיח מספיק כדי למנוע עיוותים מבניים בעת התנגשות פתאומית בקירות המבוך.
- **מודולריות וגמישות תכנונית:** העבודה המכנית הידנית עם רצועות אלומיניום העניקה לנו גמישות רבה במהלך שלבי הניסוי והפיתוח (Prototyping). היא אפשרה לנו לבצע שינויים והתאמות במיקומי הרכיבים, מרכז הכובד והחיישנים, ללא צורך בתכנון מחדש או בהדפסה חוזרת של שלדה שלמה.
- **יציבות מכנית:** קונסטרוקציית המתכת מעניקה קשיחות מבנית מוחלטת לתושבות המנועים ומגינה על רכיבי האלקטרוניקה הרגישים (כמו ה-Raspberry Pi 5 וה-Arduino Uno) מפני זעזועים מכניים.

2. מערכת ההנעה ברובוט (Propulsion Subsystem)

לצורך תמרון יעיל, מהיר ומדויק במרחב הצר של הזירה, נבחרו ארכיטקטורת ההנעה והתצורה הגיאומטרית הבאה:

תצורת גיאומטריית הגלגלים: הנעה דיפרנציאלית (Differential Drive)

הרובוט מתבסס על מערכת הנעה דיפרנציאלית המורכבת משני גלגלים מניעים עצמאיים הממוקמים בצדדיו (ציר מרכזי), וגלגל תומך חופשי (Caster Wheel) הממוקם בחלקו האחורי/קדמי לשם ייצוב סטטי ומניעת התהפכות בעת תאוצה או בלימה.

בשיטה זו, היגוי הרובוט ותנועתו במרחב הדו-ממדי מנוהלים באופן בלעדי על ידי שינוי מהירות הסיבוב וכיוון הסיבוב של כל אחד משני המנועים בנפרד (על ידי קביעת מתח ה-PWM בבקר):



- **תנועה קווית (ישר קדימה / אחורה):** שני המנועים מסתובבים באותה המהירות ובאותו הכיוון בדיוק.
- **פנייה קשתית (Curve Turn):** מנוע אחד מסתובב במהירות גבוהה יותר מהמנוע הנגדי, מה שיוצר רדיוס סיבוב התלוי בהפרש המהירויות.
- **סיבוב על הציר (Zero-Radius Turn / Spin):** מנוע אחד מסתובב קדימה בעוד המנוע השני מסתובב אחורה במהירות זהה. תכונה זו היא קריטית ובעלת חשיבות עליונה עבור תחרות הרובוט; מאחר ורוחב המסדרונות במבוך עומד על 46 ס"מ בלבד, היכולת להסתובב במקום על ציר הסיבוב הגיאומטרי של הרובוט (ICR\$ - Instantaneous Center of Rotation\$) מונעת פגיעה בקירות ומאפשרת חילוץ מהיר ממבוי סתום או תמרון בפינות חדות.

3. מערכת הכוח ובחירת מנוע (Actuators & Motor Selection)

סוג המנועים: מנועי זרם ישר עם תמסורת (DC Gearmotors)

הרובוט מצויד בשני מנועי זרם ישר (DC Motors). מנוע DC פועל על עיקרון כוח לורנץ, שבו זרם חשמלי העובר בסליל בתוך שדה מגנטי מייצר מומנט סיבובי. מאחר ומנועי DC נוטים להסתובב במהירות גבוהה מאוד (אלפי RPM) אך לספק מומנט (כוח סיבוב) נמוך במיוחד, שולבה בכל מנוע **תיבת תמסורת גלגלי שיניים (Gearbox)**.

התמסורת המכנית מפחיתה את מהירות הסיבוב של ציר המנוע ומגדילה ביחס ישר את המומנט המועבר לגלגלים, בהתאם לנוסחה ההנדסית:

$$\tau_{out} = \tau_{in} \times N \times \eta$$

(כאשר T הוא המומנט, N הוא יחס התמסורת של הגיר, ו- η מייצג את נצילות המערכת המכנית).
הגדלת מומנט זו חיונית ביותר כדי להעניק לרובוט את הכוח הנדרש להתגבר על כוח החיכוך

הסטטי ברגע הזינוק, לשאת את משקל מערכות המחשוב והסוללות, ולטפס בצורה חלקה על פני אי-רציפויות או הפרשי גובה ברצפת העץ של הזירה (שיכולים להגיע עד 5 מ"מ לפי תקנון התחרות).

ניתוח הנדסי: הנעה בחוג פתוח מפוצה תוכניתית (Open-Loop Actuation with High-Level SLAM Feedback)

מאפיין ייחודי ומתקדם ביותר בארכיטקטורה של הרובוט שלנו הוא השימוש במנועים **ללא אנקודרים פיזיים** (חיישנים המותקנים על ציר המנוע ומוודים את סיבובי הגלגל). ברובוטים פשוטים ובסיסיים,



חוסר באנקודרים מייצר בעיית ניווט קטסטרופלית, שכן לבקר אין אינדיקציה חומרית ישירה לגבי המרחק שהרובוט עבר (Dead Reckoning).

עם זאת, בארכיטקטורה המבוזרת והאקדמית שפיתחנו, הפכנו את החיסרון החומרי הזה ליתרון מערכתי, על ידי פתרון הבעיה ברמת התוכנה באמצעות **חוג בקרת מיקום עילי מבוסס לייזר ואינרציה (Laser & Inertial-Based Localization)**:

1. **הכשל המובנה באנקודרים מסורתיים:** בתנאי הזירה של הרובוט (רצפת עץ חלקה, פסי הדבקה), גלגלי הרובוט נוטים לסבול מהחלקות (Slippage) תכופות. במצב של החלקה, אנקודר פיזי ידווח לבקר שהרובוט התקדם, בעוד שבפועל הרובוט מסתובב במקום – מה שמייצר שגיאת אודומטריה מצטברת חמורה המטעה את מערכת הניווט.
2. **הפתרון התוכני במערכת שלנו:** במקום להסתמך על חישוב מיקום עיוור מתנועת הגלגלים, מחשב הקצה הראשי (**Raspberry Pi 5**) מריץ אלגוריתם **SLAM** (Simultaneous Localization and Mapping) מתקדם. האלגוריתם מקבל קלט רציף בתדר גבוה מחיישן ה-**LiDAR** ומבצע התאמת סריקות בזמן אמת (Scan Matching). הוא משווה את ענן הנקודות הדו-ממדי הנוכחי לענן הנקודות הקודם ומחשב בצורה מתמטית תוכנית את המיקום המדויק של הרובוט בזירה, ללא תלות בסיבובי הגלגלים.
3. **שילוב בקרת כיוון אינרציאלית:** כדי להבטיח בקרת כיוון (Heading) מושלמת בעת נסיעה ישר ובזמן פניות במבוך, המיקרו-בקר (**Arduino Uno**) דוגם את חיישן ה-**IMU** (מד המהירות הזוויתית - הג'ירוסקופ). הבקר מפצה על חוסר היכולת למדוד את מהירות המנועים בנפרד על ידי תיקון מתחי ה-PWM למנועים בזמן אמת, ובכך שומר על זווית הנסיעה המבוקשת בצורה אקטיבית וחסנית רעשים.



סיכום רכיבי הרובוט

על מנת לממש את הארכיטקטורה המבוצרת והאוטונומית של הרובוט, המערכת חולקה לארבע תתי-מערכות הנדסיות עיקריות: מכניקה ושלדה, מערכת כוח והנעה, מערך עיבוד ובקרה, ומערך חישה ואקטואציה אקטיבית. להלן פירוט מורחב של הרכיבים המרכיבים את המערכת ותפקידם המערכתי:

1. תת-מערכת מכניקה, שלדה וחיבורים

תת-מערכת זו מהווה את ה"שלד" הפיזי של הרובוט, האחראי על עמידה באילוצי הממדים של התחרות (31*31*31 ס"מ) ושמירה על קשיחות מבנית.

- **2 לוחות אלומיניום (20\25 times o"מ):** משמשים כבסיס השלדה הדו-שכבתית (Double-Deck Chassis). הלוח התחתון נושא את המנועים, הסוללות ורכיבי ההספק, בעוד הלוח העליון מבודד ומחזיק את בקרי העיבוד והחיישנים הרגישים.
- **6 מוטות אלומיניום:** משמשים כעמודי תמיכה ומקשרים (Spacers) בין הלוח התחתון לעליון, ומבטיחים את יציבות המבנה הדו-שכבתי.
- **גלגל כדורי (Caster Wheel):** ממוקם בחלקו האחורי/קדמי של הרובוט. משמש כנקודת תמיכה שלישית לייצוב סטטי, ומאפשר תנועה חופשית בכל כיוון ללא התנגדות מכנית בעת פניות.
- **קשיחים וחיבורים מכניים:**

- **ברגים ואומים מסוגים שונים:** משמשים לקיבוע חסון של תושבות המנועים, הבקרים והחיישנים לשלדת האלומיניום.
- **שייבות קפיץ (Split Lock Washers):** רכיב מכני קריטי המשולב תחת האומים. תפקיד שייבת הקפיץ הוא להפעיל לחץ קבוע המונע את שחרור הברגים כתוצאה מהרעידות המכניות של המנועים ופני השטח של הזירה.

2. תת-מערכת כוח, אנרגיה והנעה

מערכת זו אחראית על אספקת אנרגיה יציבה ומבודדת מרעשים לכלל רכיבי הרובוט, והמרת האנרגיה החשמלית לתנועה מכנית במרחב.

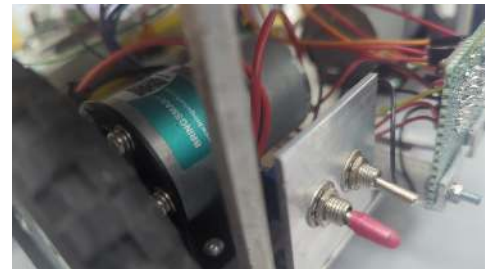
- **סוללת NiMH (ניקל-מטאל הידריד) 12V:** סוללת הכוח הראשית המיועדת להזנת רכיבי ההספק.



- **סוללת Power Bank (קיבולת 20,000mAh, הספק 22.5W, 5V/4.5A):** סוללת הגיבוי והמחשוב הראשי. מספקת זרם גבוה וציב דרך כבל USB-C ייעודי ישירות לבקר ה-Raspberry Pi 5, ומבטיחה שמחשב הקצה לא יחווה קריסות (Reset) עקב שינויי עומס במנועים.

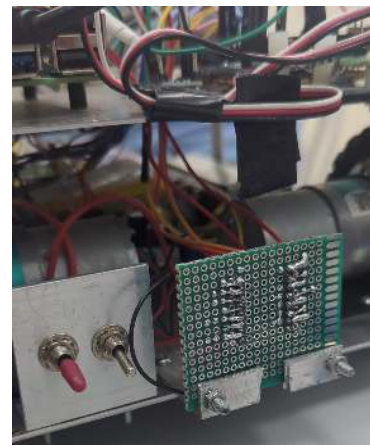


- **2 מנועי DC דגם JGB520 (מהירות 150RPM):** מנועי ההנעה הראשיים של הרובוט, הכוללים תמסורת גלגלי שיניים מובנית המורידה את מהירות הסיבוב ל-150 סל"ד לטובת הגדלת המומנט והשגת שליטה מדויקת בקצב התקדמות הרובוט.



- **2 גלגלים (קוטר 9 ס"מ):** מחוברים לצירי המנועים. קוטר הגלגל נבחר בקפידה כדי לאפשר מהירות קווית אופטימלית ומרווח גחון מספק למעבר מעל מכשולים בגובה 5 מ"מ ברצפת הזירה.

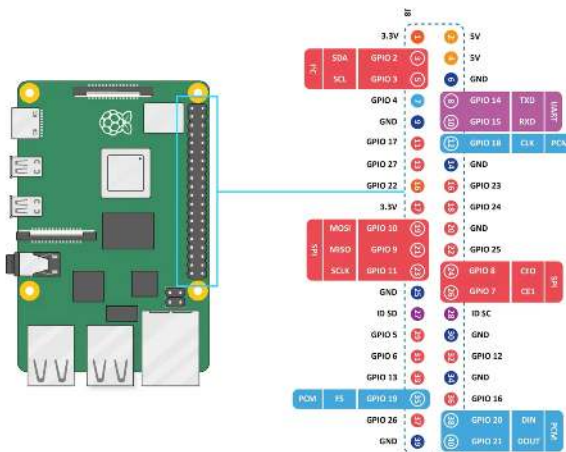
- **דרייבר מנועים (Motor Driver):** כרטיס הגשר (H-Bridge) המקבל פקודות לוגיות (PWM ו-Direction) מבקר הארדואינו, ומנתב את הזרם הגבוה מסוללת ה-NiMH למנועים לשליטה במהירות ובכיוון הנסיעה. הדרייבר (הירוק)



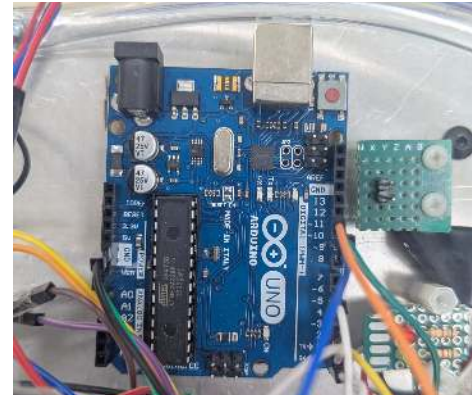
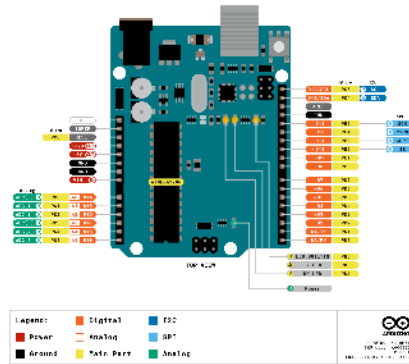
- **מפצל מתחים (5V, 8V, 12V) וממירי מתח (3.3V, 5V):** רשת חלוקת הכוח (Power Distribution Network). המפצל והממירים מורידים ומייצבים את מתחי הסוללות לרמות העבודה הנדרשות עבור רכיבי האלקטרוניקה השונים והחיישנים.
- **רכיבי מיתוג וחיווט:**
 - **מפסק מנועים ומפסק מתחים:** מפסקים פיזיים נפרדים המאפשרים ניתוק בטיחותי ומהיר של מערכת ההנעה בנפרד ומערכת הפיקוד בנפרד בזמן כיוול ובדיקות מעבדה.
 - **מחברי XT60 לסוללות:** מחברי זרם גבוה תעשייתיים המבטיחים חיבור חסון, בעל התנגדות אפסית, המונע נתקים רגעיים בעת תנועה.
 - **חוטים באזורים וצבעים שונים:** תשתית החיווט המערכתית, הצבועה לפי קוד צבעים (אדום/שחור למתחים, צבעים שונים לאותות) לשם מניעת קצרים ויעול תהליך איתור התקלות (Troubleshooting).

3. תת-מערכת עיבוד, בקרה ואקטואציה אקטיבית

- שכבת הניהול המבוזרת של הרובוט, המפרידה בין עיבוד עילי בזמן אמת לבין ניהול חומרה ישיר.
- **בקר Raspberry Pi 5:** מחשב הקצה הראשי של הרובוט (\$High-Level Processor\$). מריץ את מערכת ההפעלה אובונטו וסביבת ROS2. מופקד על משימות כבדות: עיבוד נתוני הלייזר, הרצת אלגוריתם ה-SLAM, מיפוי המבוך וקבלת החלטות הניווט האוטונומיות.



- **בקר Arduino Uno:** מיקרו-בקר ה-Embedded של הרובוט (\$Low-Level Processor\$). מחובר ל-Raspberry Pi 5 באמצעות פרוטוקול תקשורת טורי (UART). תפקידו לדגום בזמן אמת את מערך החיישנים האנלוגיים והדיגיטליים, לנהל את בקרת המנועים, ולשלוט על מנגנון הכיבוי.



- **מערכת כיבוי אקטיבית:**
 - **מאוורר (Brushed DC Motor):** האקטואטור המשמש לכיבוי פיזי של הנר באמצעות יצירת ספיקת אוויר ממוקדת.
 - **ממסר מאוורר (Relay):** מפסק אלקטרומכני המאפשר לבקר הארדואינו (הפועל בזרם נמוך) לסגור מעגל חשמלי ולהזרים זרם גבוה מסוללת ה-NiMH ישירות למנוע המאוורר ברגע זיהוי הלהבה.

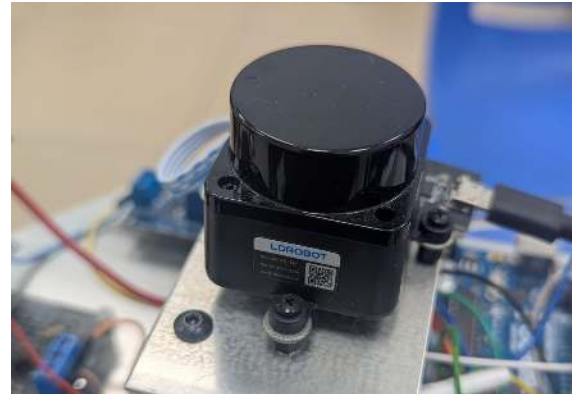
ממסר המאוורר



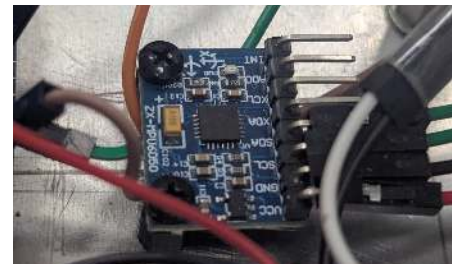
4. תת-מערכת חישה והיתוך מידע (Sensors Subsystem)

מערך העיניים והחיישנים של הרובוט, המאפשר לו לזהות את סביבתו, לקבוע את מיקומו במרחב, להזניק את עצמו ולמצוא את מוקד האש.

- **חיישן לידאר (LiDAR):** חיישן סריקת לייזר היקפי. מחובר ישירות ל-Raspberry Pi 5 ומספק ענן נקודות דו-ממדי (Point Cloud) (\$2D) של קירות המבוך בתדר גבוה, המהווה את הקלט הראשי עבור אלגוריתם ה-SLAM.



- **חיישן IMU (Inertial Measurement Unit):** יחידת מדידה אינרציאלית הכוללת ג'ירוסקופ ומד תאוצה. מחוברת לארדואינו ומספקת נתוני קצב סבסוב (Yaw Rate) לצורך שמירה על כיוון נסיעה ישר ותיקון פניות מדויק במקום.



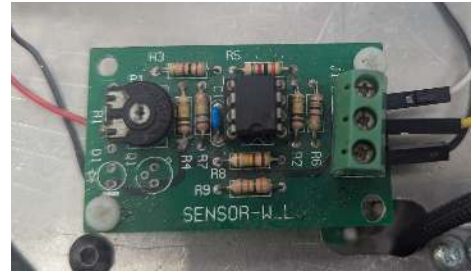
- **חיישן UV דגם Hamamatsu (אורכי גל 185-235nm):** חיישן פוטו-אלקטרי רגיש במיוחד לקרינה אולטרה-סגולה הבוקעת באופן בלעדי מלהבה חשופה. משמש לגילוי ראשוני וודאי של הנר ממרחק גדול ותוך סינון מוחלט של אור נראה.

חיישן UV





- **חיישן להבה פוטורנזיסטור (אורך גל 432nm) + דרייבר Sensor W.L:** חיישן אופטי הפועל בתחום האור הנראה/אינפרא-אדום הקרוב. משמש כחיישן מיקום מדויק (מוכוון חזיתית) כדי לאפשר לרובוט להתמרכז מול הנר במרחק קרוב רגע לפני הפעלת המאוורר.



-
- **חיישן אולטראסוני (Ultrasonic Sensor):** חיישן טווח מבוסס גלי קול. משמש ככלי גיבוי ומניעת התנגשויות (Object Avoidance) בטווח הקרוב מול קירות או מכשולים בדירה.
- **מיקרופון (Sound Sensor):** חיישן אקוסטי המשמש לקליטת צפצוף גלאי העשן בתדר הייעודי שנקבע בתקנון, לצורך ביצוע "הזנקה קולית" אוטונומית (Acoustic Start) ללא מגע יד אדם.
- **2 נורות לד (LEDs):** משמשות כאינדיקטורים חזותיים (ממשק אדם-מכונה פשוט) המדווחים על סטטוס הרובוט בזמן אמת (למשל: נורית אדומה מסמנת קליטת אות הזנקה, נורית ירוקה מסמנת זיהוי להבה וכיבוי).



מערכת החשמל והספק (Power Distribution) (System)

1. מבוא וארכיטקטורת הפרדת מסילות כוח (Power Isolation)

מערכת החשמל ברובוט האוטונומי תוכננה ונבנתה תוך התחשבות באחד האתגרים המרכזיים בהנדסת מערכות רובוטיות מורכבות: **מניעת רעשים אלקטרומגנטיים ונפילות מתח רגעיות (Brownouts)**. במערכות רובוטיות פשוטות, נהוג להשתמש במקור אנרגיה יחיד עבור כלל רכיבי הרובוט. אולם, מנועי זרם ישר (DC Motors) צורכים זרמי שיא גבוהים במיוחד (Stall Current) ברגעי הזינוק, ובזמן עבודתם הם מייצרים רעשים חשמליים חזקים (Spikes) כתוצאה מהקשת החשמלית במברשות המנוע. לרוב היינו מחברים את מחשב הקצה הרגיש, ה-Raspberry Pi 5, לאותה מסילת כוח של המנועים, שינויי המתח הללו היו גורמים לנפילת מתח רגעית מתחת לסף העבודה של המחשב, מה שהיה מוביל לאיפוס (Reset) ולקריסת מערכת ה-ROS2 בזמן תחרות.

כדי לפתור כשל מובנה זה, מומשה ברובוט ארכיטקטורה של **שתי מסילות כוח מבודדות לחלוטין** בעלות אדמה משותפת (Common Ground), כפי שמתואר בפירוט להלן:

2. מסילת כוח א': עיבוד וחישוב עילי (High-Level Computing Domain)

מסילה זו מיועדת אך ורק להזנת "מוח" הרובוט, ונדרשת לספק מתח יציב במיוחד וזרם גבוה ללא תנודות.

- **מקור האנרגיה:** סוללת גיבוי ניידת (Power Bank) בעלת קיבולת עצומה של **20,000mAh**, הסוללה תומכת בפרוטוקול טעינה מהירה ומספקת הספק של **22.5W** במתח של **5V** וזרם מקסימלי של **4.5A**.
- **נתיב הזרם:** האנרגיה מועברת ישירות מהסוללה באמצעות כבל USB-C חסון ואיכותי אל כניסת ההזנה של ה-Raspberry Pi 5.
- **תפקיד מערכתי:** הבחירה בספק כוח ייעודי זה מבטיחה כי מעבד ה-Cortex-A76 של ה-Raspberry Pi 5 יקבל את מלוא האנרגיה הנדרשת לעיבוד בזמן אמת של אלגוריתמי ה-SLAM וענן הנקודות מה-LiDAR, ללא שום תלות במצב המנועים או במטען הסוללה המשנית.

3. מסילת כוח ב': הנהגה, הספק וחישה (Actuation & Sensing Domain)

מסילה זו מנהלת את הזרמים הגבוהים והמתחים המשתנים הנדרשים להנעת המכניקה, הפעלת האקטואטורים ודגימת החיישנים.

- **מקור האנרגיה:** סוללת **NiMH** (ניקל-מטאל הידריד) **12V** המחוברת באמצעות מחבר **XT60** לזרם גבוה. סוללה זו מתאפיינת ביכולת פריקה גבוהה (C-Rating) המאפשרת לה לספק זרמי שיא למנועים ללא פגיעה בתאי הסוללה.
-



- **רכיבי מיתוג ובטיחות:** מיד לאחר הסוללה ממוקם **מפצל מתחים** ראשי המשמש כנתק חירום והפעלה של המערכת כולה.

רשת חלוקת המתחים (Power Distribution Network - PDN)

מסוללת ה-NiMH, המתח מגיע אל **מפצל מתחים ייעודי** המפצל את הכוח לשלוש מסילות עבודה עיקריות, המשמשות רכיבים שונים בהתאם למתח העבודה הנומינלי שלהם:

1. מסילת 12 וולט (12V Line):

- המתח מועבר דרך **מפסק מנועים** ייעודי (המאפשר לנתק את זרם המנועים לצורכי בטיחות וכיול קוד במעבדה מבלי לחבות את החיישנים).
- ממפסק המנועים, מתח ה-12V מזין ישירות את **הדרייבר (Motor Driver)** ואת **ממסר המאוורר (Relay)**. הדרייבר והממסר מנתבים את מתח ה-12V לרכיבי ההספק: שני מנועי ה-150RPM (JGB520) ומנוע הזרם הישר של מאוורר הכיבוי (Brushed DC Motor).

2. מסילת 8 וולט (8V Line):

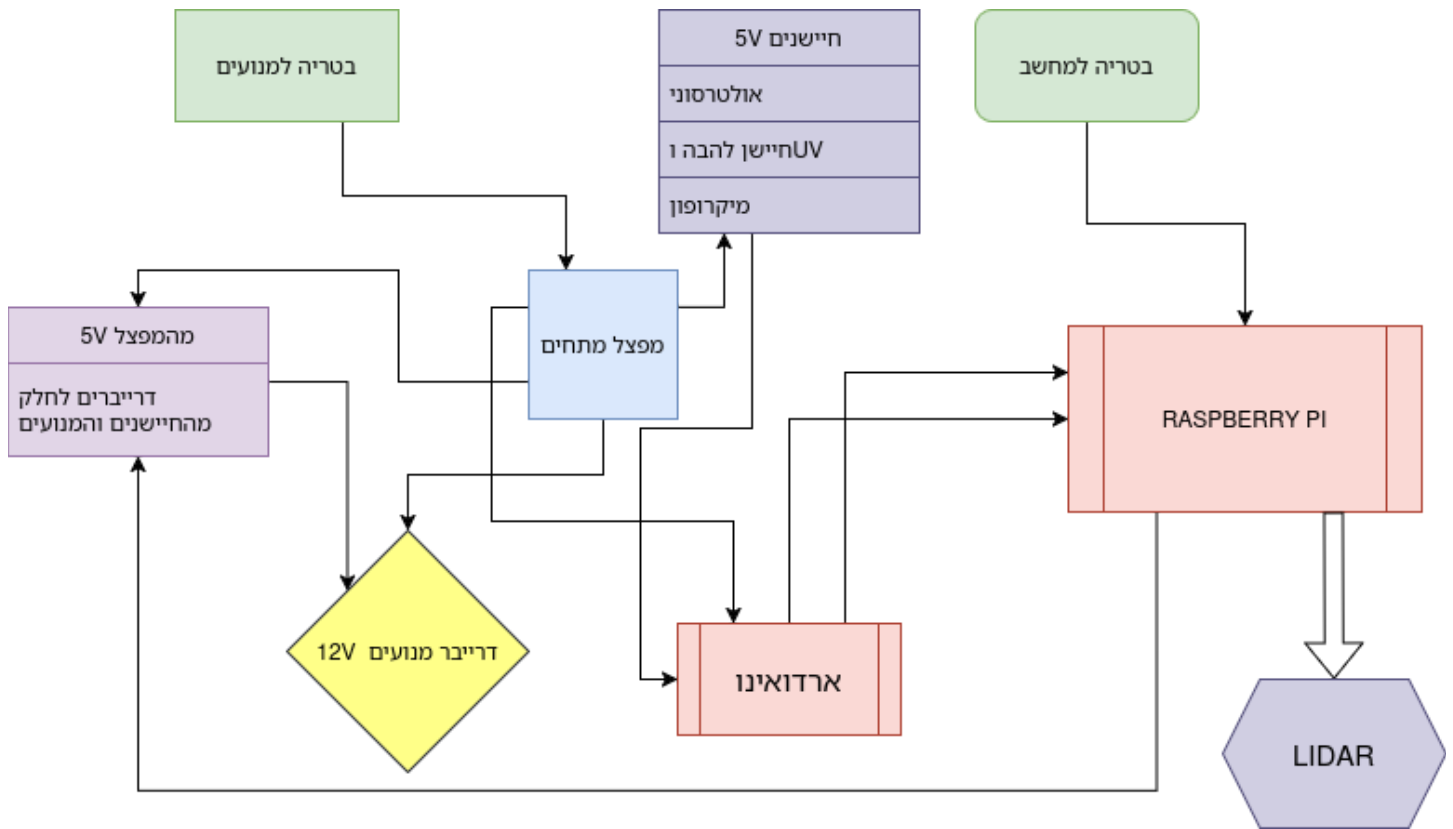
- מתח זה מנותב ישירות לכניסת ה-VIN של בקר ה-Arduino Uno. מייצב המתח הפנימי של הארדואינו מוריד את המתח ל-5V פנימי בצורה יעילה ובטוחה, ומפעיל את המיקרו-בקר.

3. מסילת 5 וולט (5V Line):

- מסילה זו מיועדת להזנת מערך החיישנים המגוון של הרובוט הדורש מתח מיוצב של 5V.
- **ממיר מתח (5V / 3.3V):** מחובר למסילה זו ומוריד את המתח ל-3.3V עבור רכיבים מבוססי לוגיקה נמוכה (כמו חיישן ה-IMU).
- מסילות ה-5V וה-3.3V מזינות את רכיבי החישה המחוברים לארדואינו: חיישן ה-UV ((Hamamatsu), החיישן האולטראסוני, המיקרופון, ה-IMU, וחיישן הלהבה (פוטוטרנזיסטור) יחד עם הדרייבר הייעודי שלו (Sensor W.L). בנוסף, מתח זה מפעיל את שתי נורות הLED המשמשות כאינדיקטורים.

4. יישום אדמה משותפת (Common Ground Reference)

אף על פי שמערכת החשמל מבוססת על שני מקורות אנרגיה נפרדים לחלוטין (ה-Power Bank וסוללת ה-NiMH), רכיבי הפיקוד חייבים לתקשר ביניהם. ה-Raspberry Pi 5 וה-Arduino Uno מחליפים ביניהם מידע דיגיטלי באמצעות פרוטוקול **UART (תקשורת טורית)**. כדי שאותות המתח הדיגיטליים (הביטים של ה-0 וה-1) יפוענחו בצורה נכונה בשני הצדדים, חובה להגדיר פוטנציאל ייחוס זהה. לשם כך, בוצע **חיבור פיזי קשיח בין קו האדמה (GND) של ה-Arduino Uno לבין קו האדמה (GND) של ה-Raspberry Pi 5**. חיבור זה, המכונה **Common Ground**, סוגר את המעגל החשמלי של קווי התקשורת, מונע קריאות שווא (Floating Pins) ומאפשר העברת נתונים חסינת רעשים במהירות גבוהה.





הסבר על חיישני הרובוט:

חיישן מרחק אולטראסוני (Ultrasonic Sensor)

- **הסבר על פעילות החיישן:** החיישן מבוסס על פליטה והחזרה של גלי קול בתדר גבוה (מעבר לטווח השמיעה האנושי, לרוב כ-40kHz). החיישן מורכב ממשדר (Trigger) ומקלט (Echo). המשדר שולח גל, הגל פוגע בעצם, חוזר, ונמדד על ידי המקלט. מרחק העצם מחושב על פי מהירות הקול באוויר וזמן הטיסה של הגל לפי הנוסחה: $Distance = (Time * Velocity) / 2$
התפקיד ברובוט: עוזר במניעת התנגשויות בטווח הקרוב. הוא מספק קריאה מהירה וישירה של מרחק מקירות או מכשולים דינמיים שהרובוט עלול לפגוש במבוך.
אינטגרציה וחיבורים: החיישן מחובר לפינים של GPIO של ה-Raspberry Pi 5 ומנוהל באמצעות קוד. הנתונים מפורסמים ב-ROS2 Topic ייעודי.

חיישן לידאר (LiDAR - Light Detection and Ranging)

- **הסבר על פעולות החיישן:** חיישן הלידאר משלב פולסי לייזר (אור אינפרה-אדום) ורכיב מכני מסתובב ב-360 מעלות. החיישן יורה אלפי פולסים של אור בשנייה ומודד את הזמן שלוקח לקרן לחזור מהעצמים סביבו (ToF). על ידי סנכרון זמן החזרת האור עם זווית הסיבוב של המנוע, החיישן מייצר ענן נקודות דו-ממדי המייצג את פני השטח במדויק.
- **התפקיד ברובוט:** זהו החיישן המרכזי המשמש עבור אלגוריתם SLAM (Simultaneous Localization and Mapping) הלידאר מאפשר לרובוט למפות את המבוך של תחרות הרובונר בדיוק של מילימטרים ולדעת את מיקומו המדויק בתוכו.
- **אינטגרציה וחיבורים:** החיישן מתקשר בחיבור USB ל-Raspberry Pi וב-ROS2 נעשה שימוש ב-Node ייעודי של יצרן החיישן, המפרסם הודעות מסוג `sensor_msgs/msg/LaserScan` ישירות לתוך קומפוננטת הניווט.



. יחידת מדידה אינרציאלית - IMU (Inertial Measurement Unit)

- **הסבר על פעולות החיישן:** רכיב אלקטרוני מסוג MEMS (Micro-Electro-Mechanical Systems) המשלב בתוכו מד-תאוצה תלת-צירי ומד-מהירות זוויתית תלת-צירי (ולעיתים גם מצפן/מגנטומטר). מד התאוצה מודד כוחות אינרציאליים הפועלים על הרובוט, והג'ירוסקופ מודד את קצב שינוי הזווית במרחב.
- **תפקיד ברובוט:** מניעת שגיאות מצטברות בתנועת הרובוט. מכיוון שהגלגלים של הרובוט עלולים להחליק על פני השטח (שגיאות אודומטריה), ה-IMU מספק קריאה בלתי-תלויה של כיוון הרובוט ($\$Yaw\$$). נתון זה קריטי כדי שהרובוט ידע אם הוא באמת פנה ב-90 מעלות במבוק.
- **אינטגרציה וחיבורים:** החיישן מתקשר עם המעבד הראשי. נתוני הזווית והתאוצה עוברים פילטור (כמו פילטר קלמן או פילטר משלים) ומפורסמים כהודעת `sensor_msgs/msg/Imu`.

חיישן להבה / אש (Flame Sensor - Phototransistor)

- **הסבר על פעולות החיישן:** חיישן זה מבוסס על רכיב אופטי מסוג פוטוטרנזיסטור הרגיש במיוחד לאורך הגל של קרינת אור אינפרה-אדום, בטווח שבין 760nm ל-1100nm. טווח זה מאפיין את קרינת הנפלטת מלהבה חשופה (כמו נר). כאשר קרינת IR פוגעת בבסיס הפוטו-טרנזיסטור, הוא מאפשר זרימת זרם חשמלי היחסי לעוצמת האש.
- **תפקיד ברובוט:** מטרת העל של רובוט הרובונר היא איתור וכיבוי אש במבוק. חיישן הלהבה משמש כ"עיניים" של הרובוט לזיהוי מיקום הנר. עוצמת האות האנלוגי/דיגיטלי המופקת מהחיישן מאפשרת לרובוט לדעת האם הוא מתקרב למקור האש או מתרחק ממנו.
- **אינטגרציה וחיבורים:** החיישן מוציא אות אנלוגי או אות דיגיטלי. האותות מנוטרים על ידי מערכת הבקרה כדי להפעיל את מצב ה"כיבוי" בתוכנה ברגע שהערך חוצה את סף המטרה.



חיישן קול / מיקרופון (Sound Detection Subsystem)

- **הסבר על פעילות החיישן:** החיישן מבוסס על מיקרופון קונדנסר קטן. המיקרופון מכיל קפסולה ובה שתי לוחיות קרובות - אחת קבועה ואחת גמישה הזזה בהתאם לגלי הקול (תנודות בלחץ האוויר). תנועת הדיאפראגמה משנה את הקיבול החשמלי בין הלוחיות, ושינוי זה מתורגם לאות מתח חשמלי חלש. אות זה עובר דרך מגבר ייעודי על גבי המודול כדי להפוך אותו לקריא עבור המעבד.
- **תפקיד ברובוט:** בהתאם לתקנון תחרות ה"רובוטר", על הרובוט לבצע הזנקה אוטונומית רק בעת שמיעת סיגנל האזעקה הרשמי. חריגה מסעיף זה גוררת פסילה. חיישן המיקרופון משמש כ"אוזניים" של המערכת בשלב ה-Pre-start. ברגע שמזוהה עוצמת קול או תדר התואמים לאזעקה, התוכנה מעבירה את הרובוט ממצב המתנה למצב ניווט פעיל.
- **אינטגרציה וחישובים:** החיישן מוציא אות אנלוגי ישירות לפין GPIO ב-Raspberry Pi 5. ברגע שעוצמת הרעש עוברת סף מסוים שנקבע על ידי הפוטנציומטר שעל המודול.



תכנות

1. מבוא

המערכת מבוססת על ROS2 ומהווה מערכת רובוטית מלאה הכוללת ניווט אוטונומי, מיפוי סביבה, זיהוי אובייקטים, תקשורת בין בקרים ושליטה במנועים.

המחשב על הרובוט מריץ מערכת הפעלה UBUNTU מלאה ועליה רצה שכבת ROS2 שמדברת עם החיישנים והמנועים ומבצעת את כל החישובים. כדי לתכנת את ה-PI התחברתי מרחוק אל המחשב דרך WIFI בחיבור SSH שנותן לי להיכנס לטרמינל ולקודד דרך vscode.

במהלך פיתוח הפרויקט נעשה שימוש במספר טכנולוגיות מתקדמות מתחום הרובוטיקה:

- ROS2 לניהול כלל המערכת
- Nav2 עבור ניווט אוטונומי
- SLAM למיפוי הסביבה
- LiDAR לסריקת מרחב וזיהוי מכשולים
- Scan Matching לשיפור הדיוק של המיקום
- State Machine לניהול מצבי פעולה
- תקשורת UART בין Raspberry Pi ל-Arduino Uno

בנוסף לכך פותחה מערכת ייעודית לזיהוי נר באמצעות שילוב של חיישן UV, פוטוטרנזיסטור וחיישן אולטרה-סוני. שילוב החיישנים אפשר זיהוי אמין יותר של מקור האש וצמצום זיהויים שגויים.



2. מושגים חשובים להבנת מערכת הניווט והמיפוי של הרובוט

Odometry (אודומטריה)

אודומטריה היא שיטה להערכת מיקום הרובוט, כיוונו ומהירותו באמצעות נתונים מחיישנים כמו אנקודרים ו-IMU. המידע מתפרסם בדרך כלל ב-Topic בשם /odom. האודומטריה מספקת מידע רציף ומהיר, אך עלולות להצטבר בה שגיאות לאורך זמן.

(TFs Transform Frames)

מערכת TF מנהלת את הקשרים בין מערכות הצירים של רכיבי הרובוט השונים. היא מאפשרת לחשב את המיקום והכיוון של כל רכיב ביחס לרכיבים אחרים. ה-Frames המרכזיים הם map, odom ו-base_link.

Scan Matching

Scan Matching הוא תהליך שבו משווים סריקות עוקבות של חיישן LiDAR כדי לחשב את תנועת הרובוט. שיטה זו משפרת את דיוק המיקום ומפחיתה שגיאות שנוצרות באודומטריה.

SLAM (Simultaneous Localization and Mapping)

SLAM הוא תהליך שבו הרובוט בונה מפה של הסביבה ובו-זמנית מחשב את מיקומו בתוכה. המערכת משלבת מידע מחיישנים שונים כדי ליצור מפה מדויקת ולשפר את יכולת הניווט.

נמשיך להרחיב על הנושאים הללו בהמשך.



3. רקע תיאורטי

ROS2

ROS2 היא פלטפורמת תוכנה לרובוטיקה המאפשרת בניית מערכות מודולריות באמצעות Nodes.

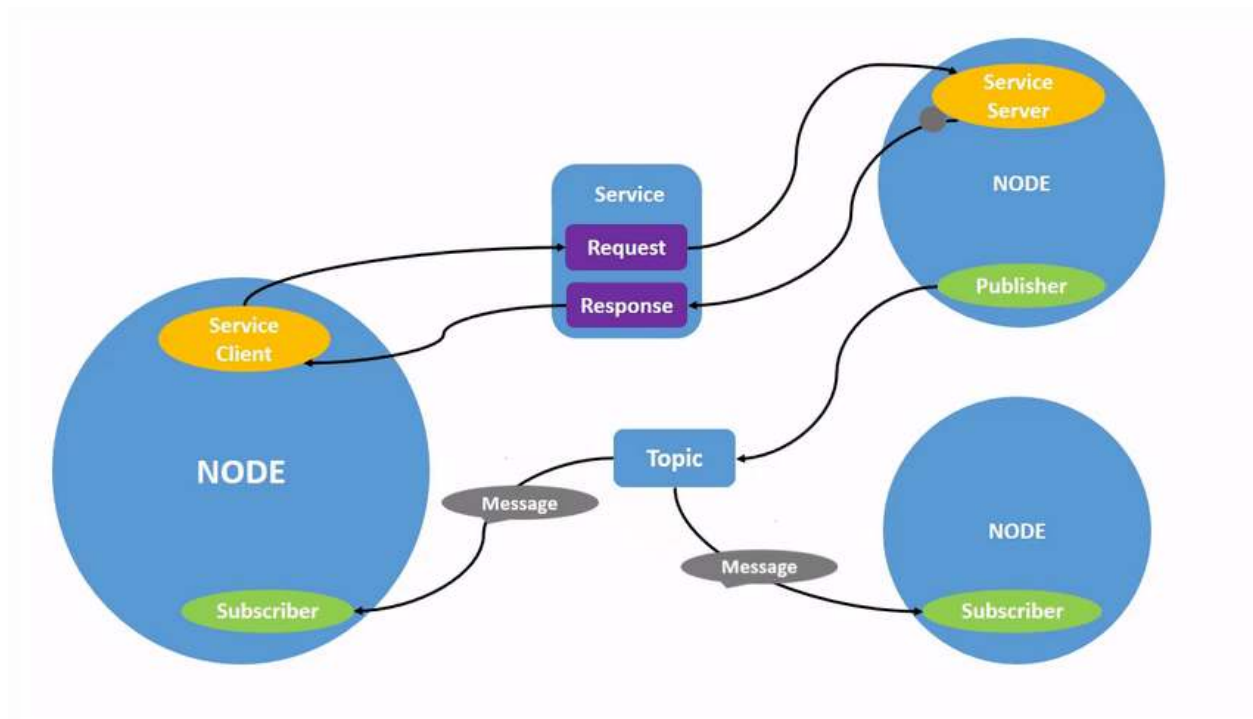
ROS 2 Services, Topics ו-Nodes במערכת ROS 2

מערכת ROS 2 בנויה ממספר רכיבים מרכזיים המאפשרים לתהליכים שונים ברובוט לתקשר זה עם זה בצורה מסודרת ויעילה. שלושת המושגים החשובים ביותר הם Nodes, Topics ו-Services.

Node (צומת) הוא יחידת תוכנה עצמאית המבצעת משימה מסוימת בתוך המערכת. כל Node אחראי על תפקיד מוגדר, כגון קריאת נתוני חיישנים, שליטה במנועים, חישוב מיקום הרובוט או יצירת מפה של הסביבה. חלוקת המערכת למספר Nodes מאפשרת פיתוח, תחזוקה ואיתור תקלות בצורה נוחה יותר, מכיוון שכל רכיב פועל באופן עצמאי.

Topic (ערוץ תקשורת) הוא מנגנון להעברת מידע באופן רציף בין Nodes שונים. Node אחד יכול לפרסם (Publish) מידע ל-Topic מסוים, ו-Node אחר יכול להירשם (Subscribe) לאותו Topic ולקבל את הנתונים. שיטה זו מתאימה במיוחד למידע המתעדכן באופן קבוע, כגון נתוני מצלמה, חיישן IMU, מדידות LiDAR, נתוני אודומטריה או פקודות תנועה. היתרון המרכזי הוא שמספר Nodes יכולים לקבל את אותו המידע בו-זמנית ללא צורך בחיבור ישיר ביניהם.

Service (שירות) הוא מנגנון תקשורת המבוסס על בקשה ותשובה. בניגוד ל-Topics, המשמשים להעברת מידע רציפה, Service מיועד לפעולות חד-פעמיות שבהן Node אחד שולח בקשה ומקבל תשובה מיידית. לדוגמה, ניתן להשתמש ב-Service כדי לאפס את האודומטריה, לשמור מפה, לבדוק את מצב הסוללה או להפעיל תהליך כיול של חיישן. מודל זה דומה לפעולה של לקוח ושרת ברשת מחשבים.



ברובוט אוטונומי טיפוס, Node אחד אוסף נתונים מחיישני הגלגלים ומפרסם אותם ב-Topic, Node אחר משתמש בנתונים אלו כדי לחשב את מיקום הרובוט ולפרסם מידע חדש ב-Topic נוסף, ו-Node

של מערכת הניווט משתמש במידע זה לצורך קבלת החלטות. במקביל, כאשר נדרש לבצע פעולה מיוחדת, מערכת הניווט יכולה לשלוח בקשת Service ל-Node המתאים ולקבל ממנו תשובה.

לסיכום, Node הוא יחידת תוכנה המבצעת משימה, Topic הוא ערוץ להעברת מידע רציף בין Nodes, ו-Service הוא מנגנון לביצוע בקשות חד-פעמיות ולקבלת תשובה. שלושת המנגנונים הללו מהווים את הבסיס לתקשורת ולשיתוף המידע במערכת ROS 2.

כל Node אחראי על פעולה מסוימת במערכת. לדוגמה:

- Node אחד שולט במנועים
- Node אחר קורא מידע מהחיישנים
- Node נוסף אחראי על ניווט



היתרונות של ROS2:

- מודולריות
- יכולת הרחבה
- עבודה בזמן אמת
- ספריות מוכנות לניווט ומיפוי

בפרויקט זה ROS2 שימשה כמערכת המרכזית של כל רכיבי התוכנה.

דוגמה לחלק מנושאי ROS במערכת שלנו :

```
gilad@gilad-HP-255-G8-Notebook-PC: ~  
/map_updates  
/mic  
/odom  
/odom_scan  
/odometry/filtered  
/parameter_events  
/plan  
/plan_smoothed  
/planned_footprints  
/planner_selector  
/planner_server/transition_event  
/pose  
/preempt_teleop  
/received_global_plan  
/robot_description  
/rosout  
/route_graph  
/route_server/transition_event  
/scan  
/scan_raw  
/set_pose  
/slam_toolbox/feedback  
/slam_toolbox/graph_visualization  
/slam_toolbox/scan_visualization  
/slam_toolbox/transition_event  
/slam_toolbox/update  
/smoothed_footprints  
/smoother_server/transition_event  
/speed_limit  
/staging_pose  
/tf  
/tf_static  
/ultrasonic  
/unsmoothed_plan  
/velocity_smoother/transition_event  
/waypoint_follower/transition_event  
gilad@gilad-HP-255-G8-Notebook-PC: ~$
```

4 המתמטיקה של ROS2 Transformations-

אחד החלקים החשובים ביותר במערכת היה ניהול המיקום והכיוון של הרובוט בתוך הסביבה.

ב-ROS2 כל רכיב במערכת מוגדר בתוך מערכת צירים משלו. לדוגמה:

- base_link מייצג את מרכז הרובוט
- laser מייצג את מיקום חיישן ה-LiDAR
- map מייצג את המפה הגלובלית
- odom מייצג את מיקום הרובוט לפי odometry

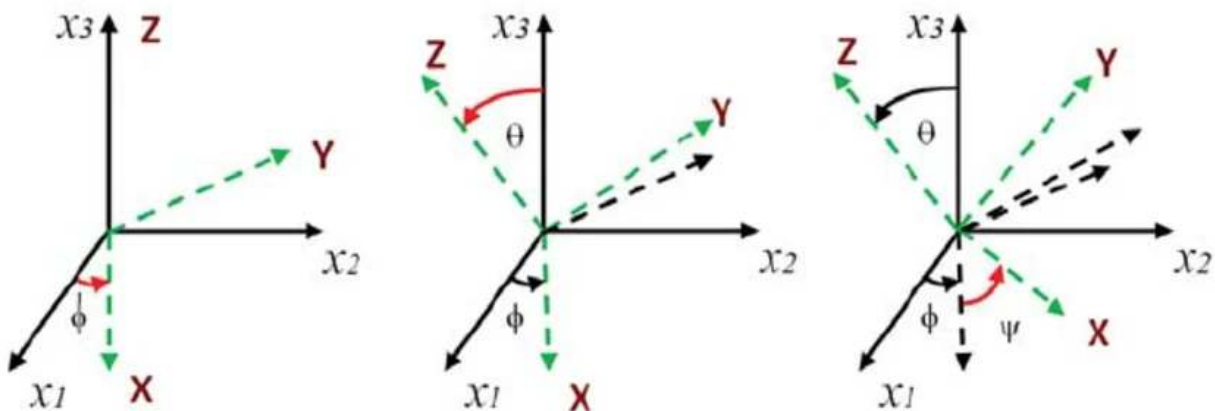
כדי שכל הרכיבים יעבדו יחד, ROS2 משתמש במערכת בשם TF.

TF מאפשר לבצע המרות בין מערכות צירים שונות.

לדוגמה:

- חיישן ה-LiDAR רואה אובייקט בזווית מסוימת
- אבל Nav2 צריך לדעת איפה האובייקט נמצא ביחס למפה

לכן המערכת מבצעת סדרת Transformations.



מערכת צירים דו-ממדית



הרובוט נע במרחב דו-ממדי ולכן כל מיקום שלו מיוצג באמצעות מערכת קואורדינטות הכוללת ערכי x , y ו- θ . ערך x מייצג תנועה קדימה ואחורה, ערך y מייצג תנועה לצדדים, ו- θ מייצג את זווית הסיבוב של הרובוט ביחס למפה. בכל רגע בזמן המערכת מחשבת מחדש את מיקום הרובוט על סמך המהירות והזווית שלו.

כאשר הרובוט מתקדם קדימה המיקום החדש שלו מחושב באמצעות פונקציות טריגונומטריות. אם הרובוט נמצא בזווית מסוימת ביחס למפה, התנועה קדימה אינה משנה רק את x אלא גם את y . לכן המערכת משתמשת בפונקציות $\sin$ ו- $\cos$ כדי לחשב את כיוון התנועה המדויק.

המשוואות המרכזיות הן:

$$x_{\text{new}} = x + v \cdot \cos(\theta) \cdot dt$$

$$y_{\text{new}} = y + v \cdot \sin(\theta) \cdot dt$$

בנוסף המערכת מחשבת את שינוי הזווית באמצעות:

$$\theta_{\text{new}} = \theta + \omega \cdot dt$$

כאשר v מייצגת מהירות ליניארית, ω מייצגת מהירות זוויתית ו- dt מייצג את הזמן שעבר בין חישוב לחישוב. חישובים אלו מבוצעים עשרות פעמים בשנייה ולכן הרובוט מסוגל לעדכן את מיקומו בזמן אמת בצורה רציפה.

חישובי המיקום האלו הם הבסיס של כמעט כל מערכת ניווט רובוטית. ללא חישובים אלו Nav2 לא היה מסוגל להבין היכן הרובוט נמצא או לאן עליו לנוע.



Odometry (אודומטריה)

אודומטריה היא תהליך שבו הרובוט מעריך את מיקומו, כיוונו ומהירותו במהלך התנועה. בפרויקט זה האודומטריה מחושבת באמצעות שילוב של נתוני Scan Matching ו-IMU.

בתחילה ניסיתי להשתמש רק במידע מה-SCAN_MATCHING אך בסופו של דבר נאלצתי להוסיף פילטר קלמן מורחב כדי לחבר את המידע ביחד מכיוון שהאודומטריה לא הייתה מדויקת ומפספסת כשהרובוט הסתובב מהר אז המידע הסיבובי לא היה אמין אז נאלצתי להתקין IMU ולקחת ממנו את המידע האודומטרי.

ה-IMU מודד את התאוצה ואת קצב הסיבוב של הרובוט ומספק מידע מהיר מאוד על שינויי הכיוון והתנועה. במקביל, אלגוריתם ה-Scan Matching משווה בין סריקות LiDAR עוקבות ומחשב כמה הרובוט התקדם ולאיזה כיוון הוא זז ביחס לסביבה.

המידע משני המקורות משולב יחד בתהליך הנקרא Sensor Fusion, שבו המערכת משתמשת ביתרונות של כל חיישן. ה-IMU מספק נתונים מהירים ורציפים, בעוד ש-Scan Matching מספק מדידת מיקום מדויקת יותר ביחס לסביבה. לאחר שילוב הנתונים נוצרת הערכת מיקום יציבה ומדויקת יותר, המתפרסמת ב-Topic בשם odom/ ומשמשת את מערכות הניווט והמיפוי של הרובוט.

מערכת TF

מערכת TF ב-ROS2 אחראית על ניהול כל מערכות הצירים של הרובוט. כל רכיב במערכת מוגדר בתוך Frame משלו ולכן יש צורך לדעת כיצד כל Frame ממוקם ביחס לאחרים.

לדוגמה:

- base_link מייצג את מרכז הרובוט
- laser מייצג את מיקום ה-LiDAR
- odom מייצג את מיקום הרובוט לפי odometry
- map מייצג את מערכת הצירים הגלובלית



כאשר ה-LiDAR מזהה נקודה כלשהי, הנקודה מתקבלת ביחס ל-frame של החיישן עצמו. אולם מערכת הניווט צריכה לדעת היכן הנקודה נמצאת ביחס למפה הגלובלית.

לכן ROS2 מבצע שרשרת Transformations:

map -> odom -> base_link -> laser

כל Transformation כולל:

- Translation
- Rotation

Translation מייצג שינוי מיקום במרחב ואילו Rotation מייצג שינוי זווית.

במונחים מתמטיים המערכת משתמשת במטריצות טרנספורמציה כדי לבצע המרות בין Frames שונים.

מטריצת סיבוב דו-ממדית נראית כך:

$$R'_\theta = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}.$$

מטריצה זו מאפשרת לקחת נקודה ממערכת צירים אחת ולהמיר אותה למערכת אחרת.

כאשר הרובוט מסתובב, גם מערכת הצירים של ה-LiDAR מסתובבת יחד איתו. TF מאפשר לכל רכיבי המערכת להבין את היחסים האלו בזמן אמת.

ללא TF מערכת הניווט הייתה מקבלת נתונים סותרים משום שכל חיישן היה עובד במערכת צירים שונה.

TF הוא אחד המרכיבים הקריטיים ביותר ב-ROS2 משום שכל מערכת הניווט, המיפוי וה-localization תלויה בדיוק של ה-transforms.



דוגמה לתיאור TF ב XML של הbase_link

```
<link name="base_link">
  <visual>
    <origin xyz="0.07 0 0.02" rpy="0 0 0"/>

    <geometry>
      <box size="0.25 0.16 0.1"/>
    </geometry>

    <material name="blue">
      <color rgba="0 0 0.8 1"/>
    </material>
  </visual>

  <collision>
    <origin xyz="0.07 0 0.02" rpy="0 0 0"/>

    <geometry>
      <box size="0.25 0.16 0.1"/>
    </geometry>
  </collision>

  <inertial>
    <origin xyz="0.05 0 -0.03"/>

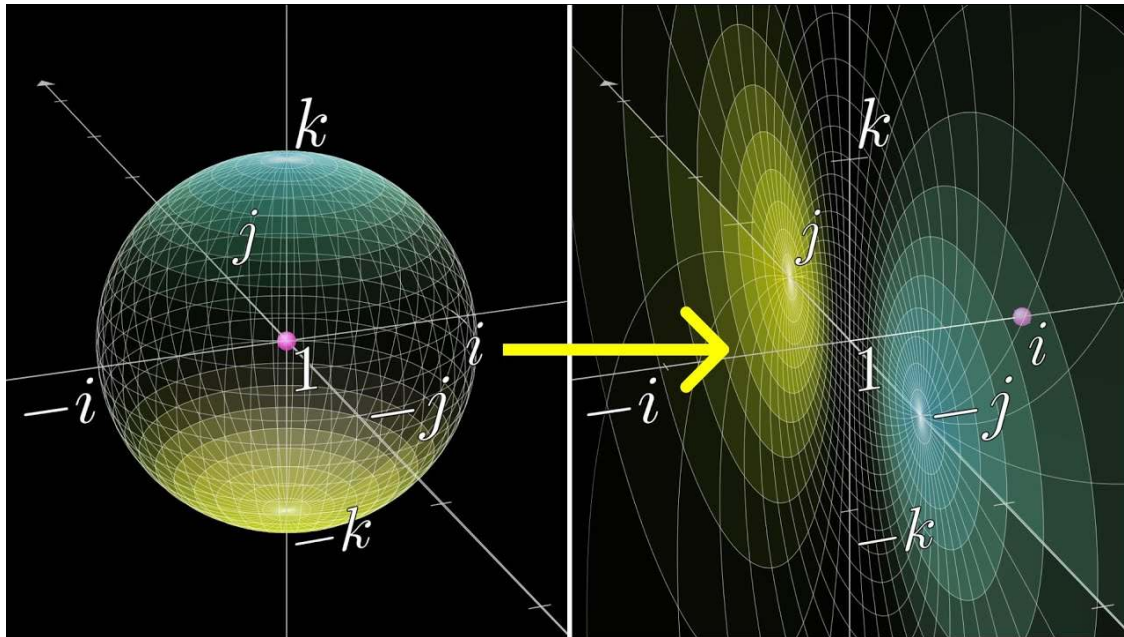
    <mass value="5.0"/>

    <inertia
      ixx="0.02"
      ixy="0"
      ixz="0"
      iyy="0.02"
      iyz="0"
      izz="0.02"/>
  </inertial>
</link>
```

Quaternion

במערכות רובוטיות משתמשים לעיתים ב-Quaternion במקום Euler Angles.

Quaternion מאפשר לייצג סיבוב בצורה יציבה יותר ומונע בעיות מתמטיות כמו Gimbal Lock.



ב-ROS2 מידע על orientation נשמר לעיתים קרובות כ-Quaternion. מערכת quaternions עובדת בעזרת כדור ארבע מימדי שמייצג סיבוב במקום להשתמש שלושה זוויות כמו $\text{euler}(\text{roll}, \text{pitch}, \text{yaw})$.

Covariance

כל חיישן כולל רמת אי-ודאות.

Covariance מייצגת את רמת הביטחון של המערכת במדידה.

לדוגמה:

- Odometry יכולה להיות מדויקת בטווח קצר
- LiDAR יכול להיות מדויק יותר לטווח ארוך

מערכת SLAM משלבת בין המדידות כדי לקבל תוצאה מדויקת יותר.



5. מבנה מערכת התוכנה

מערכת התוכנה חולקה למספר Nodes עיקריים:

1. Driver Node
2. Candle Detection Node
3. UART Communication Node
4. State Machine Node
5. Navigation Stack
6. SLAM Node

הפרדת המערכת למספר Nodes אפשרה:

- קוד מסודר יותר
- תחזוקה קלה
- איתור תקלות מהיר
- פיתוח מודולרי

Driver Node

Node זה אחראי על שליטה מלאה במערכת ההנעה של הרובוט. תפקידו המרכזי הוא לקבל פקודות תנועה ממערכת הניווט ולהמיר אותן לפעולות פיזיות של המנועים.

מערכת Nav2 שולחת פקודות תנועה דרך Topic בשם `cmd_vel`. הודעות אלו הן מסוג Twist וכוללות שני ערכים מרכזיים:

- Linear Velocity – מהירות קדימה ואחורה
- Angular Velocity – מהירות סיבוב

ה-Node Driver מאזין ל-Topic זה באופן רציף. בכל פעם שמתקבלת הודעה חדשה, ה-Node מחשב כיצד כל מנוע צריך לפעול.

כאשר הרובוט צריך לנוע קדימה:

- שני המנועים מסתובבים באותה מהירות



כאשר הרובוט צריך להסתובב:

- צד אחד מסתובב מהר יותר
- או שמנוע אחד נעצר והשני ממשיך

במערכת Differential Drive כל שינוי במהירות בין שני הצדדים יוצר סיבוב.

ה-Node גם אחראי להגבלת מהירות כדי למנוע:

- איבוד שליטה
- החלקת גלגלים
- סיבובים חדים מדי
- עומס על המנועים

בנוסף, ה-Driver Node מבצע המרה בין ערכי המהירות שמתקבלים מ-ROS2 לבין אותות PWM הנשלחים לדרייבר המנועים.

PWM מאפשר לשלוט בעוצמת המתח הממוצעת המגיעה למנועים וכך לקבוע את מהירות הסיבוב.

בזמן העבודה התגלתה חשיבות גבוהה מאוד לכיוון המנועים. במידה ומנוע אחד מוגדר הפוך, הרובוט מבצע סיבובים בכיוון שגוי למרות שכל שאר מערכת הניווט תקינה (קרה כמה פעמים).

לכן נדרש כיול של:

- כיוון מנועים
- כיוון yaw
- odometry
- orientation

ה-Driver Node פועל בקצב מהיר מאוד כדי לאפשר תגובה מיידית לשינויי מסלול.



```
class MotorDriver(Node):
    self.get_logger().info('Motor Driver Node online (using GPIO Zero)')

    def cmd_callback(self, msg):
        linear = msg.linear.x
        angular = msg.angular.z

        if angular > max_angular and linear != 0.0:
            angular = max_angular
        elif angular < -max_angular and linear != 0.0:
            angular = -max_angular

        # Differential drive math
        left_speed = linear - angular
        right_speed = linear + angular
        left_speed = left_speed * left_motor_calibration_factor
        right_speed = right_speed * right_motor_calibration_factor

        # Clamp and move left motor
        self.move_motor(self.left_motor, left_speed)
        # Clamp and move right motor
        self.move_motor(self.right_motor, right_speed)

    def move_motor(self, motor_obj, speed):
        # Ensure speed is between -1.0 and 1.0 or between the set calibration limits
        speed = max(min(speed, speed_cal), -speed_cal)
        speed = speed * -1.0

        if speed > 0:
            motor_obj.forward(speed)
        elif speed < 0:
            motor_obj.backward(abs(speed))
        else:
            motor_obj.stop()
        if abs(speed) < min_speed and speed != 0:
            speed = min_speed if speed > 0 else -min_speed

    def stop_motors(self):
        self.get_logger().info("Stopping motors and releasing GPIO...")
        self.left_motor.stop()
        self.right_motor.stop()
        self.left_motor.close()
        self.right_motor.close()
```

מה שיש בתמונה זה את הם החלקים בדרייבר שאחראים על המתמטיקה של הנעה דו גלגלי והשליחה של המידע אל הגלגלים בעזרת ספריית GPIO של ROS2



Candle Detection Node

Node זה אחראי על זיהוי הנר במהלך פעולת הרובוט. מדובר באחד החלקים החשובים ביותר במערכת משום שכל פעולת הכיבוי תלויה בדיוק של הזיהוי.

המערכת משתמשת במספר חיישנים במקביל:

- חיישן UV
- פוטורנזיסטור
- חיישן אולטרה-סוני

במקום להסתמך על חיישן יחיד, התוכנה משלבת בין הנתונים מכל החיישנים כדי לשפר את אמינות הזיהוי.

ה-Node קורא את ערכי החיישנים בצורה רציפה. לאחר קריאת הנתונים מתבצע תהליך סינון ובדיקת תנאים.

לדוגמה:

- אם חיישן ה-UV מזהה קרינה מתאימה
- והפוטורנזיסטור מזהה עוצמת אור גבוהה
- והאולטרה-סוני מודד מרחק מתאים

אז המערכת מסיקה שקיים נר מול הרובוט.

גישה זו מפחיתה False Positives.

ללא שילוב החיישנים, מקורות אור אחרים היו עלולים לגרום לזיהוי שגוי.

ה-Node מפרסם הודעות ל-State Machine כדי לעדכן האם נמצא נר.

בנוסף, המערכת משתמשת במרחק מהאולטרה-סוני כדי לקבוע מתי לעצור לפני הנר.

אם הרובוט יתקרב יותר מדי:

- המאוורר עלול לפגוע בלהבה בצורה לא יציבה
- הרובוט עלול להתנגש בנר
- חיישנים עלולים להפסיק למדוד נכון

לכן התוכנה מבצעת בדיקה רציפה של המרחק בזמן ההתקרבות.



המערכת גם כוללת מנגנון השהיה קצר כדי למנוע מעבר מהיר מדי בין מצבים כאשר הקריאות אינן יציבות.

UART From UNO Node

ה raspberry pin אינו יכול לקבל מידע מחיישנים מכיוון שהוא מריץ מערכת הפעלה מלאה (ubuntu) ולכן הוא לא מדויק מספיק כדי לקבל קריאות pwm מדויקות. כדי להתמודד עם הבעיה הזאת, הוספנו arduino uno כדי לקבל את המידע ולהעביר אותו דרך uart.

Node זה אחראי על תקשורת UART בין Raspberry Pi לבין Arduino Uno.

מטרת ההפרדה בין הבקרים היא לחלק את עומס העבודה:

- Raspberry Pi אחראי על ROS2, ניווט ועיבוד מתקדם
- Arduino אחראי על קריאת חיישנים ובקרת חומרה בסיסית

UART היא תקשורת טורית שבה מידע מועבר ביט אחרי ביט.

ה-Node פותח חיבור Serial מול Arduino ומאזין למידע באופן רציף.

כאשר Arduino שולח נתונים:

- ה-Node קורא את המחרוזת
- מפרק את המידע
- ממיר אותו לערכים מספריים
- מפרסם את הנתונים ב-ROS2 Topics

לדוגמה:

Arduino יכול לשלוח:

UV:1,LIGHT:540,DIST:18

וה-Node מפרק את המידע לערכים נפרדים.



התוכנה גם מבצעת בדיקות תקינות כדי למנוע קריסות במקרה של:

- נתונים חסרים
- רעש בתקשורת
- ניתוק UART
- תווים שגויים

במערכות רובוטיות תקשורת לא יציבה יכולה לגרום להתנהגות מסוכנת ולכן נדרשה בדיקה רציפה של תקינות המידע.

```
class SensorInputUart(Node):
    self.get_logger().info('UART reader has started')

    def read_uart(self):
        try:
            if self.ser.in_waiting > 0:
                line = self.ser.readline().decode('utf-8').strip()

                if not line:
                    return
                data = json.loads(line)

                # Ultrasonic
                msg_ultrasonic = Float32()
                msg_ultrasonic.data = float(data.get('distance', 0.0))
                self.ultrasonic.publish(msg_ultrasonic)

                # Microphone
                msg_microphone = Bool()
                msg_microphone.data = bool(data.get('microphone', False))
                self.microphone.publish(msg_microphone)

                # Flame sensor
                msg_flame_sensor = Bool()
                msg_flame_sensor.data = not bool(data.get('flame_sensor', True))
                self.flame_sensor.publish(msg_flame_sensor)

                # Fire sensor
                msg_fire_sensor = Bool()
                msg_fire_sensor.data = bool(data.get('fire_sensor', False))
                self.fire_sensor.publish(msg_fire_sensor)

        except json.JSONDecodeError:
            self.get_logger().warn(f"Invalid JSON: {line}")

        except Exception as e:
            self.get_logger().error(f"Error reading UART: {e}")
```

כאן אפשר לראות את החלק בקוד שקורא את קובץ ה JSON שמגיע דרך UART מהארדואינו. הוא לוקח אותו ושולח אתות דרך ROS2 בתור נושא שכל NODE אחר יכול לגשת אליו.



State Machine Node

State Machine Node הוא המוח הלוגי של המערכת.

במקום שכל Node יקבל החלטות לבד, State Machine מרכזת את כל מצבי הפעולה של הרובוט ומחליטה מה עליו לעשות בכל רגע.

גישה זו מונעת מצבים בהם:

- הרובוט מנסה גם לנווט וגם לכבות בו זמנית
- המנועים מקבלים פקודות סותרות
- המערכת עוברת במהירות בין מצבים

המערכת מחולקת למצבים ברורים.

מצב ראשון הוא Idle.

במצב זה הרובוט ממתין להתחלה ואינו נע.

לאחר ההפעלה המערכת עוברת ל-Exploration.

בשלב זה Nav2 ו-SLAM מתחילים למפות את הסביבה.

כאשר מתקבל מידע ממערכת זיהוי הנר, ה-State Machine עוברת למצב Searching או Approaching Candle.

בשלב ההתקרבות:

- מערכת הניווט עדיין פעילה
- המרחק מהנר נבדק
- המהירות מונמכת
- הסיבובים נהיים מדויקים יותר

כאשר המרחק מתאים:

- הרובוט עוצר
- Nav2 מפסיק לשלוח פקודות תנועה
- מופעל Relay
- המאוורר מתחיל לעבוד

לאחר הכיבוי מתבצעת בדיקה נוספת של החיישנים כדי לוודא שהנר באמת כבה.



אם עדיין מזוהה אור או UV:

- המאוורר ממשיך לפעול

אם לא מזוהה נר:

- המערכת עוברת ל-Finished

ה-State Machine משתמשת בתנאים לוגיים ברורים מאוד כדי למנוע קפיצות לא יציבות בין מצבים.

בנוסף, חלק מהמצבים כוללים timers והשהיות קצרות כדי לייצב את המערכת.



```
class StateMachineNode(Node):
    def send_goal(self, x, y):
        ..

        goal_msg.pose.header.frame_id = "map"
        goal_msg.pose.header.stamp = self.get_clock().now().to_msg()

        goal_msg.pose.pose.position.x = x
        goal_msg.pose.pose.position.y = y
        goal_msg.pose.pose.orientation.w = 1.0
        self.get_logger().info(f'Sending goal: ({x}, {y})')
        self.action_client.send_goal_async(goal_msg)

    def stop_properly(self):
        # This sends a request to Nav2 to cancel ALL active goals
        self.action_client._cancel_goal_async()
        self.get_logger().info('Sent cancel request to Nav2')

    def decision_logic(self):
        if self.state == 'Navigating':
            self.navigation()

        elif self.state == 'fire_detected':
            self.fire_detected()

        elif self.state == 'extinguishing':
            self.extinguishing()

        elif self.state == 'returning_to_base':
            self.returning_to_base()

    def fire_callback(self, msg):
        self.fire = msg.data

    def flame_callback(self, msg):
        self.flame = msg.data

    def navigation(self):
        if self.fire > 20.0:
            #turn off navigation and start flame finding behavior
            self.state = 'fire_detected'
        else: pass
```

מה שרואים כאן זו החלוקה למצבים אך עדיין לא גמרתי לכתוב את הקוד המלא של החלוקה.



6. ארכיטקטורת ROS2

מערכת התוכנה של הרובוט נבנתה בארכיטקטורה מודולרית המבוססת על ROS2. במקום לכתוב תוכנה אחת גדולה שמבצעת את כל הפעולות ביחד, המערכת חולקה למספר Nodes כאשר כל Node אחראי על משימה מסוימת. גישה זו מאפשרת הפרדה בין רכיבי המערכת והופכת את הקוד למסודר, קריא וקל יותר לתחזוקה.

כל Node פועל כתהליך עצמאי ומתקשר עם שאר הרכיבים באמצעות Topics. כאשר Node מסוים מפרסם מידע, Nodes אחרים יכולים להאזין אליו ולהשתמש בו. בצורה זו כל חלק במערכת עובד באופן עצמאי אך עדיין נשמרת תקשורת מלאה בין כל הרכיבים.

לדוגמה, מערכת הניווט מחשבת לאן הרובוט צריך לנסוע ושולחת פקודות תנועה ל-Topic בשם `cmd_vel`. ה-`Driver Node` מאזין ל-Topic זה וממיר את פקודות התנועה לשליטה אמיתית במנועים.

במקביל, Node אחר אחראי על קריאת נתוני ה-LiDAR ומפרסם אותם ל-Topic בשם `scan`. מערכת SLAM משתמשת בנתונים אלו כדי לבנות מפה של הסביבה ולעדכן את מיקום הרובוט.

ROS2 מבוססת על DDS ולכן מאפשרת תקשורת מהירה ויציבה בין תהליכים שונים. יתרון חשוב נוסף הוא האפשרות להריץ Nodes שונים על מחשבים שונים ועדיין לשמור על תקשורת מלאה ביניהם.

המערכת השתמשה במספר Topics מרכזיים:

- `scan`/ עבור נתוני LiDAR
- `cmd_vel`/ עבור פקודות תנועה
- `odom`/ עבור `odometry`
- `tf`/ עבור `transforms`
- `map`/ עבור המפה

כל Topic כולל סוג הודעה שונה. לדוגמה Topic מסוג `LaserScan` כולל מערך גדול של מרחקים שנמדדו על ידי ה-LiDAR, בעוד ש-Topic מסוג `Twist` כולל מהירות ליניארית ומהירות זוויתית.



בכל מחזור עבודה המערכת מבצעת מספר פעולות במקביל:

- קריאת חיישנים
- עדכון TF
- חישוב מיקום
- תכנון מסלול
- שליחת פקודות מנוע
- בדיקת זיהוי נר

כל הפעולות האלו חייבות לעבוד בסנכרון מדויק. אם לדוגמה מערכת ה-TF מתעכבת או שולחת מידע שגוי, Nav2 עלולה לחשב מסלול לא נכון והרובוט יתחיל להסתובב בצורה לא יציבה.

בנוסף לכך ROS2 מאפשרת שימוש ב-QoS כלומר Quality of Service. מערכת זו קובעת כיצד הודעות מועברות בין Nodes.

לדוגמה:

- האם מותר לאבד הודעות
- כמה הודעות נשמרות בזיכרון
- האם ההודעות חייבות להגיע

במערכות רובוטיות זמן אמת הוא קריטי ולכן בחירת QoS נכונה משפיעה על יציבות המערכת.

יתרון נוסף של ROS2 הוא האפשרות לבצע Debugging בצורה נוחה. באמצעות כלים כמו RViz ו-rqt ניתן לראות בזמן אמת:

- נתוני LiDAR
- מפת SLAM
- מסלולי Nav2
- TF Frames
- Costmaps
- Topics פעילים



דבר זה אפשר לזהות בעיות רבות במהלך הפיתוח כמו:

- TF הפוך
- כיוון מנועים שגוי
- רעידות בניווט
- Drift ב-odometry
- בעיות Scan Matching

7. מערכת הניווט האוטונומית

Nav2 – מערכת הניווט האוטונומית של ROS 2

Nav2 (Navigation 2) היא מערכת הניווט הרשמית של ROS 2, המאפשרת לרובוט לנוע באופן אוטונומי מנקודה אחת לנקודה אחרת תוך הימנעות ממכשולים ותכנון מסלול בטוח ויעיל. מערכת זו מהווה את "המוח" של הניווט ברובוט ומשתמשת במידע המתקבל ממערכות שונות כגון SLAM, אודומטריה, LiDAR ו-TF.

מטרתה העיקרית של Nav2 היא לקבל יעד שהוגדר על ידי המשתמש או על ידי מערכת אחרת, לחשב מסלול מתאים אל היעד, ולהפעיל את המנועים כך שהרובוט יגיע אליו בצורה בטוחה ומדויקת.

Costmap

אחד המרכיבים החשובים ביותר ב-Nav2 הוא ה-Costmap. זוהי מפה מיוחדת שבה כל אזור מקבל ערך המייצג את רמת הסיכון או הקושי לעבור דרכו.

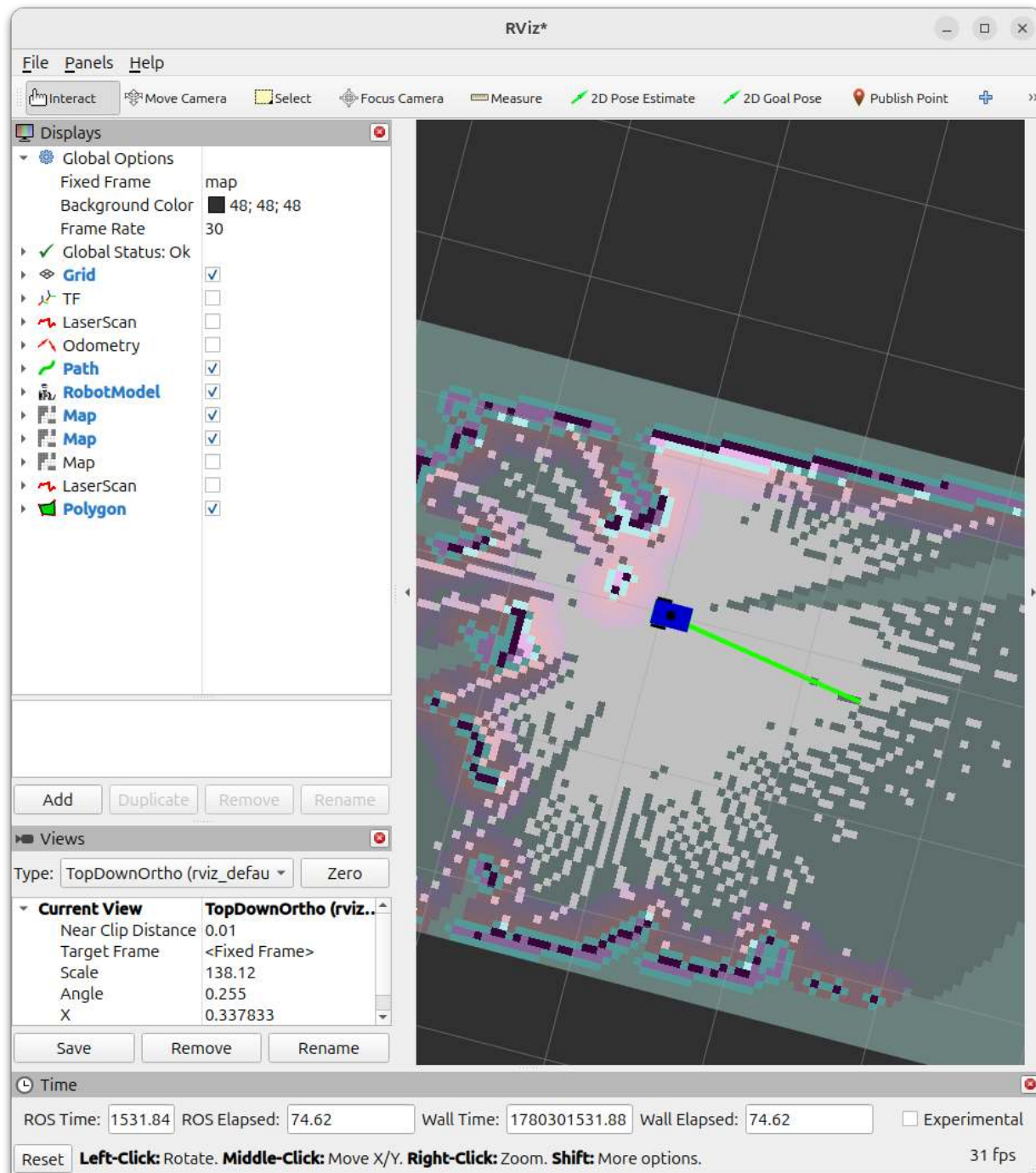
אזורים פנויים מקבלים ערך נמוך, בעוד שקירות, רהיטים ומכשולים אחרים מקבלים ערך גבוה. בדרך זו מערכת הניווט יודעת אילו אזורים בטוחים לתנועה ואילו אזורים יש להימנע מהם.

Nav2 משתמשת בדרך כלל בשני סוגי Costmaps:

Global Costmap – מייצגת את הסביבה כולה ומשמשת לתכנון מסלול ארוך טווח.



Local Costmap – מייצגת את הסביבה הקרובה לרובוט ומתעדכנת באופן רציף כדי לזהות מכשולים חדשים בזמן אמת.





Global Planner

ה-Global Planner אחראי על תכנון המסלול הכללי אל היעד. לאחר שהשתמש בוחר נקודת יעד, האלגוריתם מחשב את הנתיב הטוב ביותר על גבי המפה. המסלול מורכב מסדרת נקודות שהרובוט צריך לעבור דרכן בדרך ליעד. החישוב מתבצע תוך התחשבות בקירות, מכשולים קבועים ואזורים שאינם ניתנים למעבר. תפקידו של ה-Global Planner הוא לענות על השאלה: "איזה מסלול צריך לקחת כדי להגיע ליעד?"

(Local Planner (Controller

לאחר שנוצר המסלול, ה-Local Planner או Controller אחראי על ביצועו בפועל. מערכת זו מחשבת בכל רגע את פקודות התנועה שיש לשלוח לרובוט, כגון מהירות קדימה ומהירות סיבוב. החישובים מבוצעים באופן רציף בהתאם למיקום הרובוט, למסלול שנבחר ולמכשולים שנמצאים בסביבתו.

Obstacle Avoidance

הימנעות ממכשולים היא אחד התפקידים החשובים ביותר של Nav2. בזמן שהרובוט נע, חיישנים כגון LiDAR ממשיכים לסרוק את הסביבה ומעדכנים את ה-Local Costmap. אם מופיע מכשול חדש שלא היה קיים בזמן תכנון המסלול, המערכת מזהה אותו ומבצעת התאמות בתנועה כדי למנוע התנגשות. במידת הצורך ניתן אף לחשב מסלול חדש לחלוטין. יכולת זו מאפשרת לרובוט להתמודד עם סביבה דינמית שבה אנשים או עצמים יכולים לזוז ממקום למקום.

Recovery Behaviors

לעיתים הרובוט עלול להיתקל במצב שבו הוא אינו מצליח להתקדם, לדוגמה כאשר הדרך חסומה או כאשר החישובים אינם מצליחים למצוא נתיב מתאים.



במצבים כאלה מופעלות פעולות התאוששות (Recovery Behaviors). פעולות אלו עשויות לכלול עצירה, סיבוב במקום, נסיעה לאחור או חישוב מחדש של המסלול. מטרתן היא לאפשר לרובוט לצאת ממצבים בעייתיים ולהמשיך במשימה ללא צורך בהתערבות אנושית.

Behavior Tree

Nav2 משתמשת במנגנון הנקרא Behavior Tree לניהול תהליך קבלת ההחלטות. Behavior Tree הוא מבנה לוגי המגדיר אילו פעולות יש לבצע ובאיזה סדר. לדוגמה:

1. קבלת יעד.
2. תכנון מסלול.
3. התחלת תנועה.
4. בדיקת מכשולים.
5. ביצוע התאוששות במקרה של תקלה.
6. הגעה ליעד.

גישה זו מאפשרת לבנות מערכות ניווט מורכבות בצורה מסודרת, גמישה וקלה לתחזוקה.

Controller Server

רכיב זה אחראי על חישוב פקודות התנועה בפועל. הוא מקבל את המסלול מהמתכנן ומשתמש במידע מהחיישנים כדי לחשב את ערכי המהירות שיש לשלוח למנועים.

בסופו של התהליך הפקודות נשלחות בדרך כלל ל-Topic בשם `cmd_vel`, שמכיל את מהירות ההתקדמות והסיבוב של הרובוט.

Planner Server

רכיב זה אחראי על חישוב המסלול הגלובלי. כאשר מתקבל יעד חדש, ה-Planner Server משתמש במפה וב-Costmap כדי לחשב את הנתיב המתאים ביותר.

המסלול המחושב מועבר לאחר מכן ל-Controller לצורך ביצוע.



Navigator

ה-Navigator הוא הרכיב שמנהל את כל תהליך הניווט. הוא מקבל את היעד, מפעיל את המתכנן, מפעיל את הבקר ועוקב אחר התקדמות הרובוט עד להגעה ליעד.

בפועל הוא משמש כגורם המתאם בין כל חלקי מערכת Nav2.

Nav2 tuning

Nav2 מורכבת ממספר רכיבים פנימיים הפועלים יחד. כל רכיב אחראי על שלב אחר בתהליך הניווט.

כאשר הרובוט מקבל יעד, Nav2 מבצעת מספר פעולות:

1. בדיקת המפה
2. חישוב מסלול
3. יצירת Costmaps
4. בדיקת מכשולים
5. מעקב אחרי המסלול
6. תיקון סטיות בזמן אמת

תחילה Planner Server מחשב מסלול מהמיקום הנוכחי אל היעד. החישוב מתבצע על בסיס המפה שנוצרה על ידי SLAM. לאחר מכן Controller Server משתמש ב-Regulated Pure Pursuit כדי לגרום לרובוט לעקוב אחרי המסלול.

במקביל Recovery Behaviors מופעלים אם הרובוט נתקע או מאבד את המסלול.

מערכת Nav2 משתמשת בנתונים ממספר מקורות:

- LiDAR
- Odometry
- TF
- Costmaps
- SLAM

כל המידע הזה משולב יחד כדי לחשב תנועה יציבה.



במהלך העבודה היה צורך לבצע tuning לפרמטרים רבים במערכת Nav2. שינוי קטן בפרמטרים כמו:

- inflation radius
- max velocity
- controller frequency
- goal tolerance

השפיע בצורה משמעותית על יציבות הרובוט.

במקרים מסוימים הרובוט ביצע oscillations כלומר תיקוני כיוון מהירים וחוזרים. בעיה זו נגרמה בדרך כלל מפרמטרים אגרסיביים מדי.

לאחר tuning נכון התנועה הפכה חלקה ומדויקת יותר. לא אכניס פה את דף הפרמטרים מכיוון שהוא באורך של כ-500 פלוס שורות.



8. אלגוריתמים

Scan Matcher

Scan Matching הוא תהליך שבו המערכת משווה בין שתי סריקות LiDAR שונות כדי לחשב את תנועת הרובוט.

כאשר הרובוט נע:

- ה-LiDAR מבצע סריקה חדשה
- המערכת משווה אותה לסריקה הקודמת
- האלגוריתם מנסה למצוא כיצד הסריקות מתאימות זו לזו

אם הסריקה החדשה זזה מעט שמאלה ביחס לקודמתה, המערכת מסיקה שהרובוט זז.

כך ניתן לחשב:

- שינוי מיקום
- שינוי זווית
- תנועה יחסית

Scan Matching מאפשר לשפר משמעותית את דיוק המיקום לעומת Odometry בלבד.

במהלך העבודה Scan Matching היה חשוב במיוחד משום שגם סטייה קטנה בזווית יכולה לגרום לטעויות גדולות בניווט.

Occupancy Grid Map

SLAM יוצר מפה מסוג Occupancy Grid.

המפה מחולקת לתאים קטנים.

כל תא יכול להיות:

- פנוי
- תפוס
- לא ידוע

המערכת מעדכנת את המפה בכל פעם שמתקבלת סריקת LiDAR חדשה.



כך נבנית בהדרגה מפה מלאה של הסביבה.

תכנון מסלול

מערכת Nav2 מחשבת מסלול מהמיקום הנוכחי אל היעד.

המערכת משתמשת במידע מ:

- map
- costmap
- odometry
- TF

לאחר חישוב המסלול, Regulated Pure Pursuit עוקב אחריו.

Regulated Pure Pursuit

Regulated Pure Pursuit הוא האלגוריתם האחראי על מעקב אחרי המסלול שחושב על ידי Nav2.

במקום לנסות להגיע בדיוק לכל נקודה במסלול, האלגוריתם בוחר נקודת יעד מעט קדימה ביחס למיקום הנוכחי של הרובוט. לאחר מכן הרובוט מנסה להתקדם לכיוון אותה נקודה.

גישה זו גורמת לתנועה להיות חלקה וטבעית יותר. אם הרובוט היה מנסה לעצור בכל נקודה במסלול, התנועה הייתה מלאה ברעידות ושינויי כיוון חדים.

המערכת מחשבת באופן רציף:

- את המרחק לנקודת היעד
- את הזווית בין הרובוט למסלול
- את רדיוס הפנייה הדרוש

אם הזווית למסלול קטנה:

- הרובוט ממשיך במהירות גבוהה יחסית



אם הזווית גדולה:

- המערכת מורידה מהירות ליניארית
- מגדילה מהירות סיבוב
- ומבצעת תיקון חד יותר

האלגוריתם גם מתחשב במכשולים וב-Costmaps. כאשר הרובוט קרוב לקיר או לאובייקט מסוים, המהירות יורדת כדי לשמור על יציבות ובטיחות.

אחד היתרונות הגדולים של RPP הוא היכולת לבצע תנועה חלקה גם במסדרונות צרים ובפניות חדות.

במהלך העבודה היה צורך לבצע tuning לפרמטרים שונים של Nav2 ושל RPP כדי למנוע:

- oscillations
- overshoot
- סיבובים חדים מדי
- רעידות בזמן תיקון מסלול

שינוי קטן בפרמטרים של המהירות או ה-lookahead distance השפיע בצורה משמעותית על התנהגות הרובוט.

לאחר כוונן נכון המערכת הצליחה לבצע ניווט יציב וחלק יותר.

בחרנו ב RPP מכיוון שהוא היה הכי קל לכיוון וגם רץ הכי טוב ומתאים למקומות צרים. בעיה אחת שממש הפריעה במהלך הפרוייקט היא איך לגרום לרובוט לא להיתקע בקירות ולכן הוספנו שכבת SHIM ROTATOR מעל ה RPP כדי לנסות לתעדף סיבוב במקום על סיבוב בקשת.

```
controller server:
ros_parameters:
  controller_frequency: 10.0
  min_x_velocity_threshold: 0.001
  min_theta_velocity_threshold: 0.001
  progress_checker_plugins: ["progress_checker"]
  goal_checker_plugins: ["goal_checker"]
  controller_plugins: ["FollowPath"]

  progress_checker:
    plugin: "nav2_controller::SimpleProgressChecker"
    required_movement_radius: 0.5
    movement_time_allowance: 5.0
  goal_checker:
    plugin: "nav2_controller::SimpleGoalChecker"
    xy_goal_tolerance: 0.15
    yaw_goal_tolerance: 0.2
    stateful: true
  FollowPath:
    plugin: "nav2_rotation_shim_controller::RotationShimController"
    primary_controller: "nav2_regulated_pursuit_controller::RegulatedPursuitController"
    angular_dist_threshold: 0.3
    forward_sampling_distance: 0.5
    rotate_to_heading_angular_vel: 0.8
    max_angular_accel: 1.3
    simulate_ahead_time: 1.0
    rotate_to_goal_heading: false
    use_path_orientations: false
    desired_linear_vel: 0.15
    lookahead_dist: 0.9
    min_lookahead_dist: 0.8
    max_lookahead_dist: 1.1
    lookahead_time: 1.5
    rotate_to_heading_angular_vel: 0.8
    transform_tolerance: 0.1
    use_velocity_scaled_lookahead_dist: true
    min_approach_linear_velocity: 0.05
    approach_velocity_scaling_dist: 0.3
    use_collision_detection: true
    max_allowed_time_to_collision_up_to_carrot: 1.0
    use_regulated_linear_velocity_scaling: true
```

אלה חלק מהפרמטרים שהשתמשתי בהם ב RPP לא הצלחתי להכניס את כולם הם ממשיכים עוד כמה שורות.



9. State Machine וניהול מצבים

State Machine היא מערכת לניהול לוגיקת הפעולה של הרובוט.

מצבי פעולה:

Idle

הרובוט ממתין להתחלה.

Exploration

הרובוט נע בסביבה וממפה אותה.

Searching

הרובוט מחפש נר.

Approaching Candle

הרובוט מתקרב לנר.

Extinguishing

הרובוט מפעיל את המאוורר.

Finished

המשימה הסתיימה

יתרונות State Machine

- קוד מסודר
- מניעת התנגשויות לוגיות
- ניהול החלטות יעיל
- קלות דיבוג



10. קבצי Launch והרצת המערכת

מערכת ROS2 כוללת Launch Files.

תפקידם:

- הפעלת כל ה-Nodes
- הגדרת פרמטרים
- טעינת קונפיגורציות

היתרון:

במקום להפעיל כל Node ידנית, ניתן להפעיל את כל המערכת בפקודה אחת.

במהלך הכנת ה-LAUNCH הייתי צריך ללמוד איך להשתמש במגוון סוגי API שונים ולשלב אותם ביחד כדי להפעיל את כל המערכת. לדוגמה לקח לי בערך עשר שעות במהלך כמה ימים רק להצליח להריץ את תוכנת ה-NAV2.

```
import xacro
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, TimerAction
from launch_ros.actions import Node
from launch_ros.actions import LifecycleNode
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import EmitEvent
from launch.actions import SetAction
from launch.actions import ChangeState
from launch.actions import Transition
from launch.actions import RegisterEventHandler
from launch.event_handlers import OnStateTransition
from launch.actions import GroupAction
from launch_ros.actions import SetRemap
```

זה מגוון ה-API השונים שהשתמשתי בהם רק בקוד ה-LAUNCH מתחת לזה יש עוד כמה מאות שורות אך מקוצר המקום לא הצלחתי להכניס.

11. אינטגרציה בין רכיבי המערכת

אחד האתגרים המרכזיים בפרויקט היה אינטגרציה בין רכיבי החומרה והתוכנה.



המערכת כוללת:

- חיישנים
- מנועים
- ROS2
- Arduino
- Raspberry Pi
- מערכת ניווט
- מערכת כיבוי

כל רכיב חייב לעבוד בצורה מסונכרנת.

אם הם לא יעבדו בצורה מסונכרנת הרובוט יעבוד על מידע ישן או פגום וזה יוריד את הדיוק דרמטית. כדי לשמור על הסנכרון ביציתי מספר פעולות שונות כדי לשמור על הכל בזמנים כמו תיקונים בדרייבר של הלידאר שהחברה היאה כי הוא לא נתן קריאות עקביות וגם העברתי את הקריאות אחרי זה בסוג של מסנן שיוריד רעש מהסריקות.

סיכום: זרימת המידע המלא במערכת (Full Data Pipeline)

מערכת הרובוט פועלת באמצעות שרשרת של שלבים המעבדים את המידע המתקבל מהחיישנים ומתרגמים אותו לפעולות ניווט וכיבוי נרות. כל שלב מקבל נתונים מהשלב הקודם, מבצע עליהם עיבוד ומעביר את התוצאה לשלב הבא.

איסוף נתונים מהחיישנים

השלב הראשון במערכת הוא איסוף מידע מהסביבה ומהרובוט עצמו באמצעות מספר חיישנים.

חיישן ה-LiDAR מסוג LD19 מבצע סריקה של הסביבה באמצעות קרני לייזר ומודד את המרחק ממכשולים, קירות וחפצים שונים. המידע המתקבל מאפשר לרובוט להבין את מבנה הסביבה שסביבו.

חיישן ה-IMU מסוג MPU6050 מודד תאוצות ומהירויות זוויתיות ומספק מידע על תנועת הרובוט ועל השינויים בכיוונו.

לצורך זיהוי נרות מותקנים ברובוט חיישן UV וחיישן אור, אשר מאפשרים לזהות מקורות אור בעלי מאפיינים של להבת נר.



קליטת הנתונים במערכת ROS 2

הנתונים המתקבלים מכל חיישן נקלטים על ידי צומתי ROS 2 (ROS 2 Nodes). כל צומת אחראי על קריאת הנתונים מחיישן מסוים ופרסומם במערכת באמצעות Topics.

בשלב זה הנתונים עדיין גולמיים ולא עברו עיבוד משמעותי. מטרת שלב זה היא להעביר את המידע מהחומרה אל שכבות התוכנה של הרובוט.

חישוב אודומטריה

לאחר קליטת נתוני האנקודרים מחושב מיקומו המשוער של הרובוט באמצעות אודומטריה (Odometry).

האודומטריה מחשבת את המרחק שהרובוט עבר ואת הזווית שבה הסתובב בהתאם למספר סיבובי הגלגלים. כתוצאה מכך מתקבלת הערכה של מיקום הרובוט ושל כיוונו ביחס לנקודת ההתחלה.

למרות פשטותה, אודומטריה סובלת משגיאות מצטברות הנגרמות מהחלקת גלגלים, חוסר דיוק במנועים ותנאי שטח משתנים.

מיזוג נתונים באמצעות EKF

כדי לשפר את דיוק המיקום, משולבים נתוני האודומטריה עם נתוני ה-IMU באמצעות אלגוריתם (Extended Kalman Filter (EKF).

ה-EKF משלב את המידע משני מקורות שונים ומחשב הערכת מיקום מדויקת יותר. האנקודרים מספקים מידע טוב על מרחקי נסיעה, בעוד שה-IMU מספק מידע מדויק יותר על סיבובים ושינויי כיוון.

שילוב הנתונים מאפשר להפחית את השפעת השגיאות של כל חיישן בנפרד וליצור הערכת מיקום יציבה ואמינה יותר.

התאמת סריקות (Scan Matching)

בשלב הבא מתבצעת השוואה בין סריקות LiDAR עוקבות.



המערכת בודקת כיצד יש להזיז ולסובב את הסריקה החדשה כך שתתאים בצורה הטובה ביותר לסריקה הקודמת.

מערכת SLAM Toolbox

לאחר קבלת נתוני ה-LiDAR ונתוני המיקום, נכנסת לפעולה מערכת SLAM Toolbox.

Loop Closure ותיקון שגיאות

כאשר הרובוט חוזר לאזור שבו כבר ביקר בעבר, מערכת ה-SLAM מזהה את הדמיון בין הסריקות החדשות לסריקות הישנות.

תהליך זה נקרא Loop Closure.

לאחר הזיהוי המערכת מתקנת שגיאות שהצטברו במהלך התנועה ומבצעת אופטימיזציה של המפה ושל מיקומי הרובוט. כתוצאה מכך מתקבלת מפה מדויקת יותר.

יצירת מפת תפוסה (Occupancy Grid)

לאחר עיבוד הסריקות נוצרת מפת תפוסה דו-ממדית של הסביבה.

מפה זו משמשת את מערכת הניווט לצורך תכנון מסלולים והימנעות ממכשולים.

3.10 חקירה אוטונומית (Frontier Exploration)

בשלב החקירה האוטונומית הרובוט מחפש אזורים שעדיין לא מופו.

המערכת מאתרת גבולות בין אזורים מוכרים לאזורים לא ידועים. גבולות אלו נקראים Frontiers.

הרובוט נוסע אל אזורים אלו באופן אוטומטי עד להשלמת מיפוי הסביבה.



מערכת הניווט Nav2

לאחר שקיימת מפה והמערכת יודעת את מיקום הרובוט, מועבר המידע אל מערכת Navigation 2 ((Nav2.

מערכת זו אחראית על תכנון המסלול ועל תנועת הרובוט לעבר יעד מבוקש.

תכנון מסלול

מתכנן המסלול הגלובלי (Global Planner) מקבל את מיקום הרובוט, מיקום היעד והמפה שנוצרה.

באמצעות מידע זה הוא מחשב מסלול בטוח וקצר ככל האפשר תוך הימנעות ממכשולים.

המסלול מחושב על גבי המפה ומועבר לשלב הבקרה.

Costmaps

כדי לאפשר ניווט בטוח, Nav2 יוצרת מפות עלות (Costmaps).

בקרת תנועה RPP

הבקר המקומי (Local Controller) מקבל את המסלול המתוכנן ואת מיקום הרובוט בזמן אמת.

הבקר מחשב את המהירות הליניארית ואת מהירות הסיבוב הדרושות על מנת לעקוב אחר המסלול בצורה מדויקת.

הפקודות נשלחות אל דרייבר המנועים.

הפעלת המנועים

דרייבר המנועים ממיר את פקודות התנועה למהירויות המתאימות עבור כל אחד מהגלגלים.

המנועים מופעלים בהתאם לפקודות, והרובוט מתחיל לנוע בסביבה.



במהלך התנועה החיישנים ממשיכים לאסוף מידע חדש, והתהליך כולו חוזר על עצמו באופן רציף.

מערכת זיהוי הנרות

במקביל לכל תהליך הניווט פועלת מערכת ייעודית לזיהוי נרות.

המערכת מקבלת נתונים מחיישן ה-UV ומחיישן האור ובודקת האם מתקיימים תנאים המתאימים לזיהוי להבת נר.

כאשר מזוהה נר, מועבר המידע למערכת הניווט.

התקרבות אל הנר וכיבוי

לאחר זיהוי הנר, מערכת הניווט מחשבת מסלול אל מיקום הנר ומובילה את הרובוט אליו.

כאשר הרובוט מגיע למרחק מתאים, הוא עוצר ומפעיל ממסר (Relay) המחובר למאוורר.

המאוורר יוצר זרם אוויר המכוון לעבר הלהבה ומכבה את הנר.



רפלקציות אישיות

רפלקציה אישית - גלעד ברקוב

תפקיד בפרויקט: מהנדס תוכנה ראשי ואלגוריתמיקה (\$Low-level\$)

"כאשר יצאנו לדרך עם פרויקט ה"רובור", ידעתי מראש שתחום התוכנה יהווה את אחד האתגרים המרכזיים, אך לא שיערתי עד כמה עקומת הלמידה תהיה חדה, מרתקת ומאתגרת. בעוד שבשנים עברו תלמידים התבססו על סביבות פיתוח פשוטות וחד-שכבתיות, הצוות שלנו קיבל החלטה אסטרטגית להרים את הפרויקט לרמה אקדמית ותעשייתית. החלטה זו דרשה ממני לצלול ישירות לעולם של מערכות הפעלה מורכבות, ללמוד לעבוד בסביבת אובונטו לינוקס (Ubuntu Linux) ולשלוט בארכיטקטורת ROS2 (Robot Operating System) ברמה גבוהה במיוחד.

המעבר לכתיבת קוד המשלב שפות תכנות שונות – Python למשימות לוגיקה וניהול קשרים, לצד ++C המאפשרת ביצועים מהירים בזמן אמת – דרש ממני משמעת עצמית גבוהה ושעות רבות של קריאת תיעוד טכני מורכב ופיתוח פתרונות עצמאיים. במהלך העבודה חוויתי לא פעם רגעים של תסכול, במיוחד בשלבים שבהם נדרשתי לבצע אינטגרציה ולפתור שגיאות (Debugging) במערכת ניהול ההודעות המבוססת של ROS2, או כאשר ניסיון לייצב את חוג בקרת המיקום העילי מול חיישן ה-LiDAR וה-SLAM ללא שימוש באנקודרים פיזיים.

אחד האתגרים הגדולים בתוכנה היה ניהול פרוטוקול התקשורת הטורית (UART) מול המיקרו-בקר המשני (Arduino Uno) שעליו הופקד חברי לצוות ראובן. היכולת לתקשר בין שני הבקרים, לפענח את פקטים של נתוני החיישנים המגיעים מהארדואינו בתדר גבוה ולתרגם אותם לפקודות תנועה וניווט רציפות ב-Raspberry Pi 5, דרשה מאיתנו שיתוף פעולה הדוק וסנכרון מלא. הפרויקט הזה שינה לחלוטין את התפיסה שלי לגבי הנדסת תוכנה. למדתי שמתכנת רובוטיקה אינו פועל בחלל ריק – עליו להבין את אילוצי החומרה הפיזיים ואת האלקטרוניקה שמסביבו. אני מסיים את הפרויקט הזה בתחושת סיפוק אדירה, עם ארגז כלים הנדסי ומעשי שישרת אותי ללא ספק בהמשך דרכי המקצועית והאקדמית."



רפלקציה אישית - אביתר מוצ'ניק

תפקיד בפרויקט: מהנדס מכניקה, מערכת החשמל והאינטגרציה החומרית

"פרויקט ה"רובונר" היווה עבורי מסע הנדסי יוצא דופן, אשר תבע ממני להוציא אל הפועל תכנון עמוק, יסודי ומדויק משלב הרעיון התיאורטי ועד לרובוט פיזי ומתפקד במבוך. כמי שהופקד על בניית הרובוט מאפס, נדרשתי לפתח ולבצע אינטגרציה מורכבת ביותר בין שלוש מערכות קריטיות: המבנה המכני (השלדה), מערכת החשמל הדינמית ומערכת החיישנים והבקרים. השאיפה שלנו לבנות רובוט מתקדם ורב-עוצמה יצרה אילוצים תכנוניים קשיחים, במיוחד לאור מגבלות הנפח והמידות המחמירות של תקנון התחרות (31*31*31 ס"מ).

אחת החוויות המשמעותיות והמאתגרות ביותר שחוויתי במהלך הפרויקט הייתה חוסר הוודאות והשינויים הדינמיים בחומרה. במהלך הניסויים בשטח, הגענו למסקנה שהחיישנים הקיימים אינם מספקים את רמת הדיוק הנדרשת, ונאלצנו להזמין מספר פעמים רכיבים וחיישנים מתקדמים נוספים מהאינטרנט. בעולם המכניקה, הוספה של רכיב חדש אינה עניין של מה בכך; היא 'החריבה' פעם אחר פעם את התכנון הראשוני שלנו. נדרשתי לחשב מחדש את מרכז הכובד של הרובוט, למצוא מקום פיזי לחיישנים, ולתכנן מחדש את מערכת חלוקת החשמל ומפצלי המתחים (12V, 8V, 5V) כדי למנוע רעשים אלקטרומגנטיים שיפגעו בבקרים.

שלב הביצוע הפיזי דרש ממני רמת דיוק עילאית. עבודה עם רצועות אלומיניום, חיתוך במספרי פח וקדיחת חורים בזהירות מרובה מבלי לפגוע ברכיבים אלקטרוניים יקרים ומבלי להחליש את השלדה, לימדו אותי מהי עבודת כפיים הנדסית יסודית. מעבר לעבודה הטכנית, תפקידי דרש סנכרון מתמיד והדוק עם גלעד וראובן. הייתי חייב לוודא שהמבנה שאני בונה משרת בצורה מושלמת את קוד ה-ROS2 של גלעד ואת מערך קריאת החיישנים בארדואינו של ראובן. התהליך הזה לימד אותי שיעור לחיים בניהול פרויקטים, גמישות מחשבתית תחת לחץ והתמודדות עם תקלות בלתי צפויות. אני גאה בתוצר הסופי המורכב שהרמנו יחד בשתי ידינו.



רפלקציה אישית - ראובן למברג

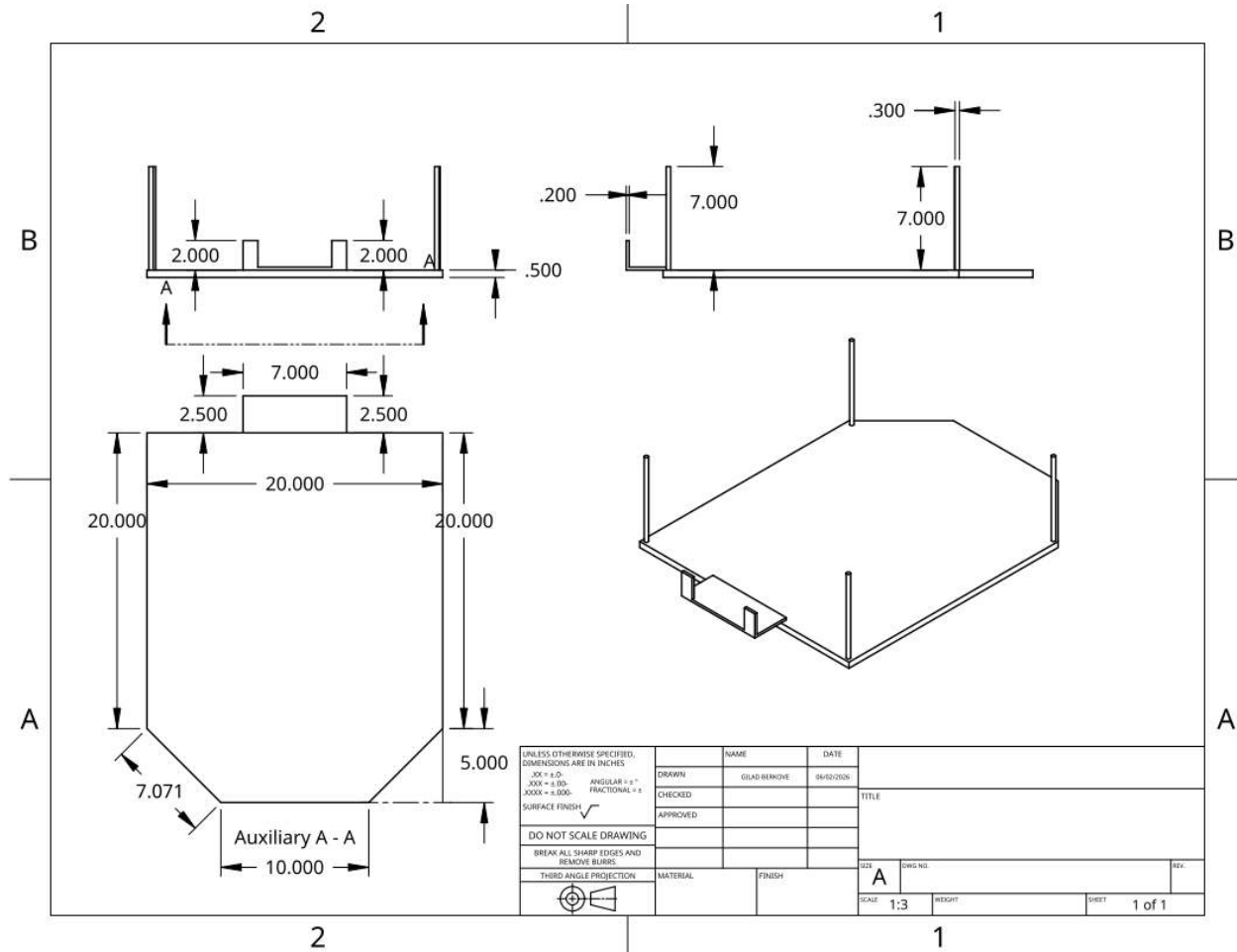
תפקיד בפרויקט: מהנדס חומרה ומתכנת מערכות קצה בזמן אמת (\$Low-Level\$)

"כאשר ניגשנו לתכנן את ארכיטקטורת המחשוב של הרובוט, הבנו מהר מאוד שמחשב הקצה הראשי (Raspberry Pi 5) יהיה עמוס במשימות כבדות של עיבוד נתוני ה-LiDAR והרצת אלגוריתמי ה-SLAM והניווט. מתוך חשיבה מערכתית מתקדמת, בחרנו ליישם ארכיטקטורה מבוזרת והתפקיד שלי בצוות היה לקחת אחריות על דרג החומרה והבקרה בזמן אמת (\$Low-Level\$) באמצעות בקר ה-Arduino Uno. משימה זו העמידה בפניי אתגר כפול: גם תכנות חומרתי מדויק בשפות ++C/C וגם הבנה פיזיקלית עמוקה של פרוטוקולי חישה.

במסגרת תפקידי, הייתי האחראי הבלעדי על החיווט, הכיול והתכנות של מערך החיישנים המגוון של הרובוט – למעט ה-LiDAR, שהתחבר ישירות ל-Raspberry Pi 5 בשל נפח הנתונים העצום שלו. נדרשתי לכתוב קוד יעיל וחסין רעשים לקריאת נתונים סימולטנית מחיישנים הפועלים בשיטות שונות לחלוטין: חיישן הטווח האולטרסוני, חיישן ה-UV לאיתור להבה, חיישן מיקום הנר האופטי ומד התאוצה והג'ירוסקופ (חיישן ה-IMU). האתגר הגדול ביותר בקוד היה לבצע סינון רעשים אנלוגיים ודיגיטליים מהחיישנים, כדי להבטיח שהמידע המועבר למערכת הניווט יהיה אמין ומדויק ב-100%.

ה'לב' של העבודה שלי היה פיתוח גשר התקשורת הטורית (UART) שחיבר בין הארדואינו ל-Raspberry Pi 5. נדרשתי לעבוד בשיתוף פעולה הדוק מאוד עם גלעד, מתכנת ה-High-Level, כדי להגדיר פרוטוקול הודעות קשיח ומהיר. הייתי צריך לוודא שהארדואינו דוגם את ה-IMU והחיישנים, אורז את המידע למחרוזות נתונים אחידות, ומשדר אותן בצורה אמינה מבלי לאבד מידע בדרך. בנוסף, העבודה המשותפת עם אביתר על פריסת החיישנים הפיזית בשלדה וההתמודדות עם רכיבים חדשים שהוזמנו והצריכו שינויי קוד תכופים, דרשה ממני סבלנות רבה ודינמיות. הפרויקט הזה העניק לי הבנה עמוקה בממשק שבין תוכנה לחומרה (Embedded Systems), ולימד אותי כיצד שיתוף פעולה, חלוקת סמכויות מקצועית ותקשורת קבוצתית טובה, הם המפתח להצלחה של כל מערכת הנדסית מורכבת."

ניתוח הנדסי והסבר מבני: שרטוט מספר 1 – הקומה הראשונה (פלטת הבסיס וההנעה)



1. מבוא לקריאת השרטוט ומפרט גיאומטרי

המסמך המובא להלן מציג שרטוט הנדסי רשמי ומקצועי (Mechanical Engineering Drawing) של פלטת הבסיס המרכזית, המהווה את הקומה הראשונה והתשתית של המערכת הרובוטית האוטונומית. השרטוט בוצע על ידי חבר הצוות גלעד ברקוב בתאריך 02/06/2026, והוא ערוך בסטנדרטים אקדמיים ותעשייתיים מקובלים, כולל שימוש בהטלות גיאומטריות (Orthographic Projections), מבטי חיתוך ופירוט מידות מדויק (Dimensioning).

מבחינה מבנית, פלטת הבסיס מבוססת על לוח אלומיניום מלבני בעל מידות נומינליות של 20.00 אינץ' אורך על 7.00 אינץ' רוחב (כ-50.8 ס"מ אורך על 17.78 ס"מ רוחב). הלוח מתוכנן בצורה המנצלת



בצורה מקסימלית את מגבלות הנפח הקשיחות של תקנון התחרות, תוך שמירה על פרופיל צר המאפשר תמרון במסדרונות הצרים של הזירה, שרוחבם עומד על 46 ס"מ בלבד. עובי הפלטה, כפי שמשתקף בפרטי המבט הצידי, מתוכנן לעמוד בעומסים מכניים קשיחים ובמומנטי כפיפה הנוצרים ממשקל הרכיבים המותקנים עליה.

בשרטוט ניתן לראות שלושה מבטים עיקריים המאפשרים הבנה תלת-ממדית מלאה של החלק:

- **מבט על (Top View):** מציג את פריסת שטח הפנים של הקומה הראשונה, מיקומי הקידוחים המרכזיים, וחלוקת האזורים הגיאומטריים.
- **מבט צידי וחזיתי:** מציגים את עובי החומר, גובה המדרגות המכניות והתמיכות המבניות.
- **מבט חיתוך ייעודי (Auxiliary Section A-A):** חיתוך עזר בקנה מידה מוגדל המציג פירוט פנימי של תושבת מכנית קריטית ברוחב 10.00 אינץ' וגובה 5.00 אינץ'. חיתוך זה מאפשר להבין את השקעים המכניים המיועדים לעיגון רכיבי הכוח המסיביים.

כלל המידות בשרטוט מיושרות לפי קווי ייחוס קשיחים (Datums) כדי להבטיח הדירות (Repeatability) ודיוק מקסימלי בעת תהליך הייצור המכני הידני שבוצע על ידי אביתר באמצעות קדיחה וכיפוף רצועות האלומיניום.

2. ארכיטקטורת המבנה הדו-שכבתי ושיקולי מרכז כובד (Center of Mass)

הבחירה לבנות את הרובוט במבנה מבוזר בעל שתי קומות נפרדות אינה החלטה אסתטית, אלא פתרון הנדסי מהותי לשתי בעיות קריטיות: יציבות דינמית בעת תנועה, ובידוד רעשים חשמליים.

הקומה הראשונה, המוצגת בשרטוט זה, מוגדרת כ"קומת הכוח וההספק" (Power & Actuation Deck). פלטה זו נושאת את כל הרכיבים בעלי המסה המשמעותית ביותר ברובוט: המנועים, הדרייברים, וסוללת ה-NiMH הכבדה. על פי חוקי הפיזיקה והמכניקה, מיקום הרכיבים הכבדים ביותר בשכבה התחתונה, קרוב ככל הניתן לפני הקרקע (הרצפה), מבטיח כי **מרכז הכובד של הרובוט (Center of Gravity - CoG)** יהיה נמוך במיוחד.

מרכז כובד נמוך מעניק לרובוט שני יתרונות דינמיים עצומים בזמן הניווט במבוך:

1. **מניעת התהפכות בתאוצה ובלימה:** מאחר ומנועי ה-JGB520 מספקים מומנט גבוה, זינוק פתאומי קדימה או בלימת חירום מול קיר היו עלולים לייצר מומנט התהפכות (Pitch Moment). בזכות מרכז הכובד הנמוך שנקבע בקומה הראשונה, הרובוט נשאר יציב לחלוטין על פני הקרקע.
2. **דיוק באודומטריה וניווט:** כאשר מרכז הכובד ממורכז ונמוך, העומס האנכי (Normal Force) המתחלק על שני הגלגלים המניעים הוא שווה וקבוע. דבר זה מקטין באופן דרמטי את תופעת ההחלקה של הגלגלים (Wheel Slippage) על רצפת העץ החלקה, ובכך משפר את אמינות הניווט ומאפשר לאלגוריתם ה-SLAM בקומה השנייה לקבל נתונים יציבים יותר.

בנוסף, המבנה הדו-שכבתי מייצר בידוד פיזי. רכיבי האלקטרוניקה הרגישים ביותר (כמו מעבד ה-Raspberry Pi 5) מורחקים לקומה השנייה, ובכך מוגנים מפני חום הנוצר מהמנועים ומפני השדות המגנטיים (EMI) שיוצרים זרמי הדרייבר בקומה הראשונה.



3. פירוט רכיבי הקומה הראשונה, פריסתם והתפקיד המערכתי של כל חלק

הפריסה המכנית של הקומה הראשונה תוכננה באסטרטגיית חלוקת משקל חכמה וזרימת אותות אופטימלית, החל מהחלק האחורי של הפלטה ועד לחלק הקדמי ביותר. להלן ניתוח של הרכיבים הממוקמים בקומה זו ותפקידם:

א. החלק האחורי: דרייבר המנועים (Motor Driver) והספק

בנקודה האחורית ביותר של השלדה ממוקם דרייבר המנועים הראשי (גשר H). מיקומו מאחור נועד לקצר את אורך החוטים המובילים ישירות אל הסוללות ואל המנועים. קיצור חוטים זה הוא קריטי ברובוטיקה, שכן חוטים ארוכים נושאים זרם גבוה פועלים כאנטנות המפיצות רעשים אלקטרומגנטיים לכל עבר. הדרייבר מקבל פקודות לוגיות במתח נמוך מהארדואינו שבקומה השנייה, ומתרגם אותן לזרם גבוה מסוללת ה-12V, אותו הוא מנתב ישירות למנועים הסמוכים אליו.

ב. צידי החלק האחורי: שני מנועי (JGB520 (150RPM וגלגלי 9 ס"מ

בצדדים הימני והשמאלי של החלק האחורי, מותקנים שני מנועי הזרם הישר עם תיבות התמסורת המובנות. מיקום המנועים והגלגלים הגדולים (קוטר של 9 ס"מ) בחלק האחורי מגדיר את גיאומטריית ההנעה הדיפרנציאלית של הרובוט.

- **למה הם בנויים ככה?** תצורה זו מאפשרת לרובוט להשתמש במנועים כציר הסיבוב המרכזי שלו. כאשר הגלגלים מסתובבים בכיוונים מנוגדים, הרובוט מסתובב סביב עצמו בדיוק מוחלט (Zero-Radius Turn). היכולת הזו חיונית כדי לצאת ממבוי סתום במבוך מבלי להתנגש בקירות העץ, ומבלי להזדקק לרדיוס סיבוב רחב שאינו קיים בזירה. התמסורת המכנית (Gearbox) מפחיתה את מהירות המנוע ל-150 סל"ד, מה שמבטיח כוח דחף (מומנט) מספק לעלייה מעל פסי הדבקה והפרשי גובה של עד 5 מ"מ ברצפה.

ג. מרכז הפלטה: מפצל המתחים הראשי (12V, 8V, 5V)

במרכז הגיאומטרי של הקומה הראשונה ממוקם מפצל המתחים הראשי של הרובוט (ה-Power Distribution Board). המרכז הוא המיקום האופטימלי ביותר עבור רכיב חלוקת כוח, מכיוון שהוא מהווה את "תחנת הממסר" האמצעית ממנה יוצאים חוטים לכל כיווני הרובוט.

- המפצל מקבל את המתח הגולמי של 12 וולט מסוללת ה-NiMH, ומפצל אותו בצורה מיוצבת ומאובטחת: מתח 12V ישיר לדרייבר ולממסר המאוורר, מתח 8V המועבר מעלה להזנת ה-Arduino Uno, ומתח 5V מיוצב המזין את רשת החיישנים. מיקומו במרכז מקטין את ההתנגדות החשמלית של החוטים ומבטיח אספקת מתח שווה לכל אגפי הרובוט.

ד. החלק הקדמי-מרכזי: מערך הסוללות (Power Bank ו-NiMH)



יותר לכיוון החלק הקדמי, אך עדיין בתוך גבולות מרכז המסה, ממוקמות שתי הסוללות של הרובוט: סוללת ה-12V (NiMH) המיועדת למנועים, וסוללת ה-Power Bank המסיבית (20,000mAh, 22.5W) המיועדת להזנת ה-Raspberry Pi 5.

- **למה הם בנויים ככה?** הסוללות הן הרכיבים הכבדים ביותר במערכת. מיקומן במרכז הפלטה, מעט קדימה מהציר המניע האחורי, מבטיח חלוקת משקל מאוזנת בין שני הגלגלים המניעים לבין הגלגל הכדורי התומך (Caster Wheel). איזון זה מונע מצב שבו הרובוט "מתיישב" על החלק האחורי ומאבד אחיזה מקדימה, או לחילופין סובל מעומס יתר בחלק הקדמי שמקשה על הפניות. הפרדת הסוללות הפיזית והחשמלית כאן מבטיחה יציבות אנרגטית מוחלטת למחשב הראשי.

ה. החלק הקדמי ביותר (חזית הרובוט): חיישן ה-UV (Hamamatsu 185-235nm)

בנקודה המתוחה והקדמית ביותר של קומת הבסיס, מותקן חיישן ה-UV הייעודי לאיתור להבת הנר.

- **למה הוא בנוי ככה?** חיישן ה-UV פועל על בסיס קליטת קרינה אולטרה-סגולה בתדרים צרים מאוד, אשר נפלטים באופן טבעי מלהבה חשופה ואינם קיימים בתאורת חדר רגילה. כדי שהחיישן יוכל "לראות" את זירת הניסוי בצורה נקייה וללא חסימות מכניות, הוא חייב להיות ממוקם בחזית הפתוחה של הרובוט, ללא שום רכיב של השלדה שיסתיר את שדה הראייה שלו (Field of View - FoV). המיקום החזיתי מאפשר לרובוט לזהות את הנר מהרגע הראשון שהוא פונה לכיוון החדר, ובכך להתחיל את תהליך ההתקרבות והכיבוי בתוך זמן קמינימלי, בדיוק כפי שנדרש בתקנון התחרות.

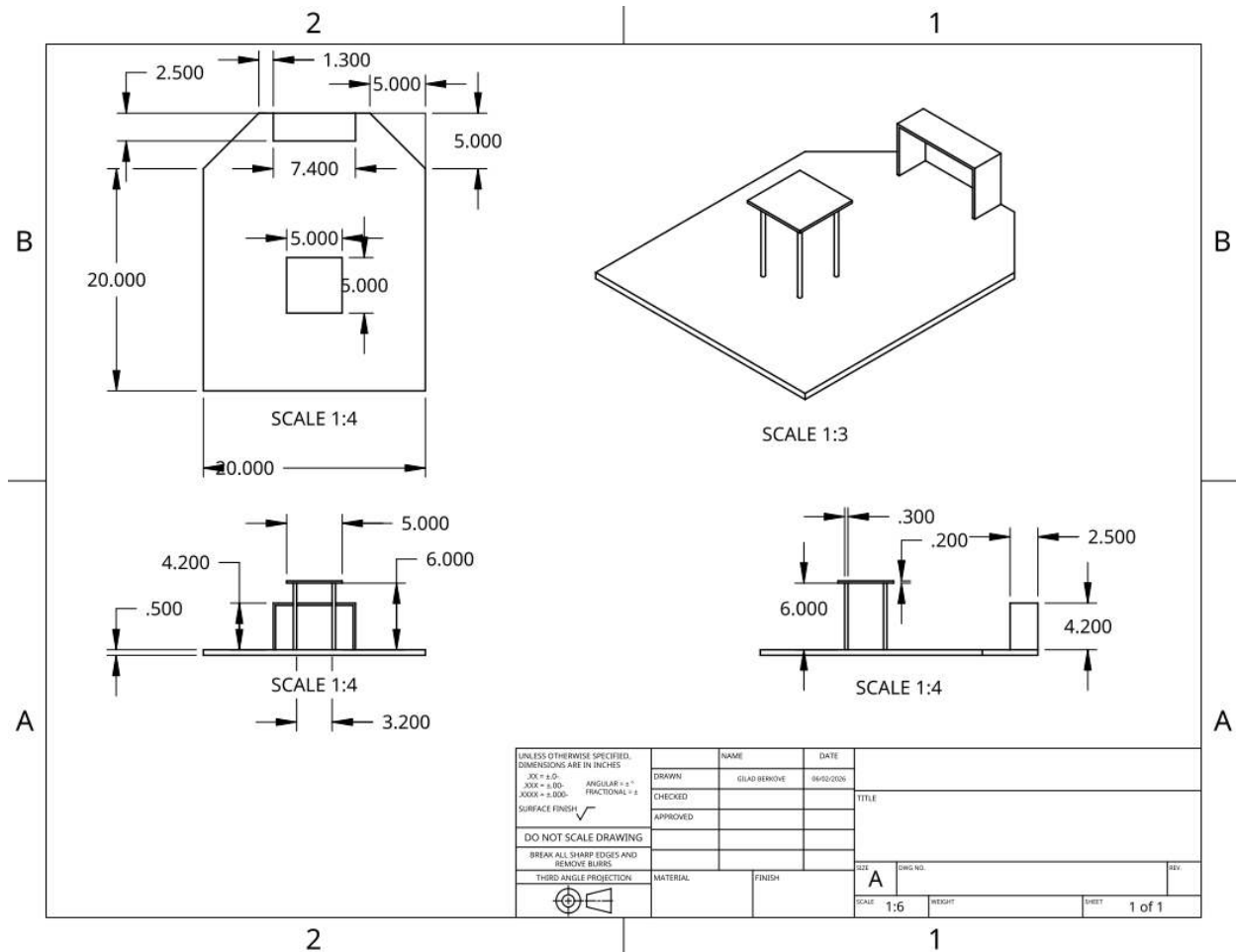
4. שיקולי הרכבה, קשיחים ומניעת כשלים מכניים

מבט העל בשרטוט מציג מערך קידוחים מדויק המיועד להרכבת רכיבי השלדה. חלק מרכזי מההצלחה המכנית של הקומה הראשונה טמון בשימוש בברגים, אומים, ושייבות קפיץ (Split Lock Washers) מתאימות.

במהלך תנועת הרובוט בזירה, מנועי ה-DC הפועלים ב-150RPM מייצרים ויברציות ורעידות מכניות מתמדות בתדרים משתנים. רעידות אלו, יחד עם המעבר מעל פסי ההדבקה והקיטועים ברצפת העץ (בגובה של עד 5 מ"מ), מייצרים מאמצים מחזוריים (Fatigue & Vibrational Stress) על כלל חיבורי הברגים ברובוט. בעולם ההנדסה, ללא אמצעי הגנה, רעידות אלו גורמות לתופעה של שחרור עצמי של האומים (Self-Loosening), מה שעלול להוביל להתפרקות תושבות המנועים או החיישנים בעיצומה של הריצה האוטונומית.

כדי למנוע כשל מכני קטסטרופלי זה, שולבו בכל נקודות העיגול בקומה הראשונה **שייבות קפיץ**. שייבת הקפיץ חתוכה בנקודה אחת ומכופפת בצורה סלילית קלה. בעת סגירת האום, השייבה נלחצת ומפעילה כוח קפיצי אנכי קבוע ומנוגד כנגד פני השטח של האום והבורג. כוח אקטיבי זה מגדיל באופן משמעותי את כוח החיכוך הסטטי בין כריכות התברג, ובכך נועל את האום במקומו בצורה חסינת רעידות ומבטיח את שרידותו המכנית של הרובוט לאורך כל שלבי התחרות והבדיקות.

ניתוח הנדסי והסבר מבני: שרטוט מספר 2 – הקומה השנייה (פלטת הפיקוד, החישה והכיבוי)



1. מבוא לקריאת השרטוט ומפרט גיאומטרי של הקומה השנייה

השרטוט המובא להלן מציג את המפרט המכני הרשמי של פלטת המחשוב והחישה העליונה (Upper Control Deck), המהווה את הקומה השנייה של הרובוט האוטונומי. השרטוט נערך ותוכנן על ידי גלעד ברקוב בתאריך 02/06/2026 בקנה מידה משתנה (מבטים כלליים ב-1:4 ומבטים ממוקדים ב-1:3 וב-1:6) כדי להעניק פתרון עיצובי מלא לצפיפות הרכיבים הגבוהה המאפיינת שכבה זו.



מבחינה גיאומטרית, הפלטה מבוססת על מבנה מורכב יותר מפלטת הבסיס התחתונה. בעוד שאורכה הכללי נשאר 20.00 אינץ' כדי לשמור על אחידות מבנית עם הקומה הראשונה, הרוחב משתנה וכולל תתי-אזורים ייעודיים. בשרטוט ניתן לראות בבירור כי החלק האחורי רחב ומציג משטח יציב של 6.00 אינץ' הנועד לשאת את שני מחשבי העיבוד, בעוד שבחלק הקדמי מבוצעת "מדרגה גיאומטרית" המצמצמת את הרוחב ל-4.20 אינץ' ויוצרת בליטה חזיתית נמוכה יותר (רוחב 2.50 אינץ' ואורך של 1.30 אינץ' בחלק המתוח ביותר).

הטבלה והמבטים ההנדסיים בשרטוט מציגים את מערך קווי הקיבוע וההטלות המשמשים לעיגון הבקרים והחיישנים, תוך הגדרת מרחקי בטיחות קבועים למניעת מגע מכני או קצרים חשמליים בין רכיבי הלוגיקה העדינים לבין עצמם.

2. הניתוח הפיזיקלי של מיקום חיישן ה-LiDAR והצורך בשדה ראייה חופשי (Field of View)

בנקודה הגיאומטרית הגבוהה ביותר של הקומה השנייה, ובחלק המרכזי שלה, ממוקם חיישן ה-LiDAR היקפי. מיקום זה מהווה את ההחלטה הארכיטקטונית החשובה ביותר במבנה המכני של הרובוט, והיא נשענת על עקרונות פיזיקליים ואלגוריתמיים קשורים.

חיישן ה-LiDAR פועל על עיקרון "זמן טיסה" (Time of Flight - ToF). החיישן מכיל דיודת לייזר המסתובבת במהירות גבוהה ושולחת פולסים של אור אינפרא-אדום בלתי נראה לכל עבר (בזווית של 360 מעלות). פולסים אלו פוגעים בקירות המבוק, מוחזרים לחיישן, ועל ידי מדידת הזמן שחלף בין שליחת האות לקליטתו, החיישן מחשב את המרחק המדויק לכל נקודה ובונה "ענן נקודות" (2D Point Cloud) של הסביבה. ענן נקודות זה מועבר ישירות למערכת ההפעלה ROS2 הרצה על ה-Raspberry Pi 5, והוא חומר הגלם הבלעדי שמשמש את אלגוריתם ה-SLAM לצורך מיפוי המבוק וקביעת מיקום הרובוט ללא אנקודרים.

למה הרובוט בנוי ככה ואסור שמשוהו יסתיר ללידאר? אם רכיב כלשהו של הרובוט - כגון חוט חשמל, בורג, מאוורר הכיבוי או שלדת האלומיניום עצמה - היה גבוה יותר ממישור הלייזר של ה-LiDAR, קרן הלייזר הייתה פוגעת ברכיב הפנימי של הרובוט במקום בקירות המבוק. פגיעה כזו מייצרת שתי בעיות קטסטרופליות: א. חסימה (Blind Spots): באותה זווית שבה הרכיב מסתיר, הרובוט הופך ל"עיוור" לחלוטין ולא מסוגל לראות אם יש קיר במרחק מטר או סנטימטר ממנו. ב. עיוות ה-SLAM (Scan Matching Failure): אלגוריתם ה-SLAM משווה כל סריקה לסריקה הקודמת כדי לחשב תנועה. אם החיישן יקלוט באופן קבוע עצם במרחק של 5 סנטימטרים (הרכיב המסתיר), האלגוריתם יתפרש זאת בטעות כאילו הרובוט עומד צמוד לקיר או יחוה שגיאות מתמטיות קשות שיגרמו לקריסת מערכת הניווט.

לכן, ה-LiDAR מוקם בטופוגרפיה הגבוהה ביותר, וכל שאר רכיבי הרובוט הונמכו בצורה מכנית מבוקרת אל מתחת למישור הסריקה שלו, מה שמבטיח שדה ראייה פתוח, נקי וחופשי ב-360 מעלות.

3. פריסת הרכיבים בחלק הקדמי (בליטת הכיבוי והחישה הממוקדת)

כדי לאפשר ל-LiDAR ראות מושלמת, תוכננה בחזית הפלטה בליטה מכנית נמוכה יותר באופן משמעותי. אזור חזיתי זה מוקדש כולו למשימת איתור המיקום המדויק של הנר וביצוע הכיבוי הפיזי:



- **חיישן הלהבה (פוטורנזיסטור 432nm) בצד שמאל:** ממוקם בחזית הצידי השמאלית של הבליטה. חיישן זה מכויל לאורך גל האופייני לפליטת האנרגיה של אש גלויה בתחום האור הנראה. בעוד שחיישן ה-UV בקומה הראשונה מבצע זיהוי ראשוני מרחוק, חיישן הלהבה החזיתי בקומה השנייה משמש לחוג בקרה קרוב - הוא עוזר לרובוט לבצע תמרון התמרכזות (Alignment) מושלם מול החדר שבו נמצא הנר.
- **המאוורר (Brushed DC Motor) בצד ימין:** ממוקם ישירות לצד חיישן הלהבה, בחזית הימנית הנמוכה. מיקום זה מבטיח כי ברגע שחיישן הלהבה מודיע שהרובוט ממרכז מול האש, המאוורר נמצא בקו ישר ומקביל לנר. הפעלת המאוורר מייצרת ספיקת אוויר ממוקדת וחזקה המכבה את הנר במהירות. בזכות ההנמכה המכנית שלו, זרם האוויר פוגע בדיוק בגובה הלהבה, והמאוורר עצמו אינו מפריע לקרן הלייזר של ה-LiDAR שמעליו.

4. פריסת רכיבי העיבוד, הניהול והתקשורת (מרכז וחלק אחורי)

משטח העבודה המרכזי והאחורי של הקומה השנייה מאכלס את רשת המחשוב המבוזרת ואת החיישנים האינרציאליים והאקוסטיים של המערכת:

- **צד שמאל מאחורה – Raspberry Pi 5:** מחשב העל של הרובוט ממוקם באגף השמאלי האחורי. מיקומו נבחר כדי לאפשר חיבור ישיר וקצר בכבל נתונים אל ה-LiDAR הממוקם במרכז, וכדי להרחיק אותו ככל הניתן ממנוע המאוורר החזיתי המייצר רעשים חשמליים. ה-Raspberry Pi 5 מריץ את אובונטו לינוקס ו-ROS2 ומנהל את המשימות האלגוריתמיות הכבדות.
- **צד ימין מאחורה – Arduino Uno:** מיקרו-בקר ה-Embedded ממוקם באגף הימני האחורי. חלוקה זו (שמאל וימין מאחור) מייצרת איזון משקלים סימטרי מושלם בירכתי הרובוט. הארדואינו מנהל את קריאת החיישנים הפיזיים ואת בקרת המנועים הנמוכה.
- **בין הבקרים – מתאם רמות מתח (5V to 3.3V Converter):** ממוקם בדיוק בתווך שבין ה-Raspberry Pi 5 לבין הארדואינו. מיקום זה אינו מקרי; הרכיב משמש כ"גשר דיגיטלי" עבור תקשורת ה-UART הטורית שביניהם. מאחר והארדואינו פועל בלוגיקה של 5V בעוד ה-Raspberry Pi 5 פועל בלוגיקה רגישה של 3.3V, חיבור ישיר ביניהם היה שורף את פני המחשב הראשי. המתאם מנמיך ומעלה את מתחי האותות הדיגיטליים בזמן אמת, ומיקומו במרכז מקצר את פסי התקשורת ומונע השראות מגנטיות ורעשים בביטים.
- **ממשק קצה – נורות הלד (LEDs):** ממוקמות ממש מאחורה בצד ימין, מחוברות לארדואינו ומאפשרות למפעיל האנושי לקבל אינדיקציה חזותית מיידית על מצב הריצה והתוכנה של הרובוט מבלי להסתכל על מסך מחשב.

5. מערך החישה הפנימי ורכיבי ההספק המשניים

בחלל המרכזי שבין הבקרים מאחור לבליטה המכנית מלפנים, ממוקמים חיישני הניווט, ההזנקה וכרטיסי ההפעלה:

- **מרכז-קדימה – חיישן ה-IMU:** יחידת המדידה האינרציאלית (הג'ירוסקופ ומד התאוצה) ממוקמת קרוב ככל הניתן למרכז הגיאומטרי של הפלטה. בפיזיקה וברובוטיקה, מיקום חיישן IMU במרכז הסיבוב של השלדה הוא קריטי; אם החיישן היה ממוקם באחד הקצוות, פנייה של



- הרובוט הייתה מייצרת תאוצה צנטריפוגלית מדומה (Centripetal Acceleration) שהייתה מעוותת את קריאות מד התאוצה. מיקומו במרכז מבטיח קריאה נקייה של קצב הסבסוב (Yaw Rate) ומאפשר לארדואינו לשמור על זווית נסיעה ישרה לחלוטין ולתקן סטיות בזמן אמת.
- **בחזית המרכזית – המיקרופון (Sound Sensor):** ממוקם בחלק הקדמי, מיד אחרי ה-IMU. מיקום זה בחזית החשופה מבטיח קליטה אקוסטית אופטימלית של צפצוף גלאי העשן (התדר שנקבע בתקנון התחרות) ומאפשר לרובוט לבצע את ה"הזנקה הקולית" האוטונומית בצורה אמינה וללא חסימות גלי קול.
 - **אגף קדמי שמאלי – דרייבר הפוטוטרנזיסטור (Sensor W.L):** ממוקם בצד שמאל של המיקרופון, בצמוד לחיישן הלהבה החזיתי. כרטיס זה מהווה את החומרה שמפעילה ומגבירה את האות האנלוגי החלש שמייצר הפוטוטרנזיסטור בעת חשיפה לאש, וממיר אותו לאות דיגיטלי נקי שהארדואינו יכול לפענח בשברירי שנייה.
 - **אגף קדמי ימני – ממסר המאוורר (Relay):** ממוקם בצד ימין של המיקרופון, בצמוד למאוורר הכיבוי. הממסר פועל כמפסק אלקטרומכני. כאשר הארדואינו מזהה הגעה לנר, הוא שולח אות פיקוד של 5V בזרם נמוך אל הממסר. הממסר נסגר (Switching) ומאפשר לזרם הגבוה והרועש מסוללת ה-NiMH שבקומה הראשונה לזרום ישירות למאוורר. מיקום הממסר בקומה השנייה קרוב למאוורר מונע את הצורך בהרצת חוטים ארוכים הנושאים זרמי מנוע גבוהים, ובכך מגן על שאר החיישנים הרגישים מפני הפרעות והרעשים החשמליים (Electrical Noise Isolation) של אקטואטור הכיבוי.



מקורות

[ROS 2 Documentation: Jazzy documentation](#) JAZZY וויקיפדיה

[Nav2](#) NAV2 וויקיפדיה

[Pulse-width modulation - Wikipedia](#) PWM וויקיפדיה

[DTOF LIDAR LD19 - Waveshare Wiki](#) LD19 לידאר

[GitHub - SteveMacenski/slam_toolbox: Slam Toolbox for lifelong mapping and localization in potentially massive maps with ROS](#) GITHUB SLAM מקורות

[GitHub - ros-navigation/navigation2: ROS 2 Navigation Framework and System](#) nav2 git

[GitHub - mertgulerx/frontier_exploration_ros2: Modern C++ frontier exploration package for autonomous mobile robots, with a core C++ library for custom integrations alongside ROS 2 Jazzy / Nav2 support.](#) Frontier exploration for robot

[AlexKaravaev/ros2_laser_scan_matcher: Laser scan matcher ported to ROS2 · GitHub](#)

icp and scan matching

[GitHub - cra-ros-pkg/robot_localization](#) ekf filter for odometry

[Getting Ready to build a ROS robot](#) ros2 tutorial

<https://robotica.co.il/rules.php> the rules for the competition

[YouTube](#) youtube

[r/ROS](#) ros subreddit